

Learning Transferable Topology Priors for Multi-Agent LLM Collaboration Across Domains

Taolin Zhang¹, Zijie Zhou², Jiuheng Wan¹, Tingyuan Hu³, Chengyu Wang⁴, Xiaofeng He³ *Member, IEEE*, and Richang Hong¹ *Senior Member, IEEE*

Abstract—Large language model (LLM)-based multi-agent systems have shown strong potential for complex reasoning by coordinating specialized agents through structured communication. However, existing topology-evolution methods typically construct or optimize a collaboration topology for each query from scratch, leading to substantial online search overhead, high inference-time token consumption, and limited scalability in multi-domain settings. We propose **TopoPrior**, a framework for learning transferable topology priors for multi-agent LLM collaboration across domains. Rather than repeatedly searching for effective collaboration structures online, **TopoPrior** learns reusable topology priors from reference collaboration graphs collected offline from multiple domains and uses them to generate query-conditioned initial collaboration graphs for downstream refinement. By shifting part of topology search from per-query online optimization to offline prior learning, **TopoPrior** amortizes search cost while remaining compatible with existing topology-evolution backbones. Technically, **TopoPrior** contains two key components. First, a transferable topology prior learning module employs a conditional variational graph framework to capture reusable structural regularities across domains in a latent space. Second, a query-conditioned latent adaptation module introduces adversarial alignment to reduce unnecessary domain discrepancy while preserving query-relevant structural variation. Experiments on multi-domain reasoning benchmarks show that **TopoPrior** consistently improves several heterogeneous topology-evolution backbones while reducing online inference-time token usage, with only modest additional trainable parameters. These results suggest that transferable topology initialization is an effective and lightweight mechanism for improving the efficiency of multi-agent LLM collaboration across domains.

Index Terms—Large language models, multi-agent systems, transferable topology prior learning, topology initialization, topology evolution, multi-domain reasoning.

I. INTRODUCTION

LARGE language model (LLM)-based multi-agent systems have emerged as a promising paradigm for complex reasoning by coordinating multiple specialized agents through structured communication. This paradigm is particularly appealing in multi-domain settings, such as healthcare [1], science [2], [3], and law [4], [5], where different queries may

require different combinations of expertise and collaboration patterns. A central challenge in this setting is how to *reuse* effective collaboration structures across domains, so that multi-agent systems can adapt to new tasks without repeatedly searching for communication topologies from scratch.

Existing approaches to multi-domain LLM reasoning can be broadly grouped into three categories, each with different trade-offs in flexibility, effectiveness, and computational cost. *Training-free prompting* methods adapt frozen LLMs through prompt engineering, few-shot demonstrations, and chain-of-thought reasoning [6]–[8]. Although easy to deploy, their performance is ultimately limited by the capability of the underlying model [9]. *Fine-tuning*-based methods specialize models to target domains through parameter updates [10], [11], but they may suffer from catastrophic forgetting and introduce substantial training and storage overhead when many domains are involved [12], [13]. By contrast, *multi-agent collaboration* improves reasoning by decomposing tasks across specialized agents and organizing their interactions through communication topologies. Recent work has progressed from static topologies to dynamic graph construction, including reinforcement-learned, pruning-based, and autoregressive approaches [14]–[17]. However, most of these methods treat topology construction as a query-level optimization problem and often perform search from scratch, which can incur substantial online overhead, accumulated communication noise, and high inference-time token consumption in multi-domain settings [18]; see Fig. 1.

This limitation motivates a different perspective: *instead of treating every query as a new topology-search problem, can we learn reusable collaboration patterns offline and use them to initialize downstream topology evolution?* To this end, we propose **TopoPrior**, a framework for learning transferable topology priors for multi-agent LLM collaboration across domains. The key idea is to shift part of the topology-search burden from per-query online graph construction to offline cross-domain prior learning. Rather than evolving a collaboration graph from scratch for every incoming query, **TopoPrior** learns reusable topology priors from reference collaboration graphs collected across multiple domains, and then uses these priors to generate query-conditioned initial collaboration graphs for downstream refinement. In this way, **TopoPrior** amortizes part of the topology-search cost across queries and domains while remaining compatible with existing topology-evolution backbones.

TopoPrior contains two key components. **(1) Transferable Topology Prior Learning.** We develop a conditional

¹ Taolin Zhang, Jiuheng Wan, and Richang Hong are with the Hefei University of Technology, Hefei 230002, China (e-mail: tlzhang@hfut.edu.cn; wan_jiuheng@163.com; hongrc@hfut.edu.cn).

² Zijie Zhou is with the China University of Petroleum (Beijing), Beijing 102249, China (e-mail: zjzhouzh@gmail.com).

³ Tingyuan Hu and Xiaofeng He are with the East China Normal University, Shanghai 200062, China (e-mail: 10245102409@stu.ecnu.edu.cn; hexf@cs.ecnu.edu.cn).

⁴ Chengyu Wang is with the Alibaba Group, Hangzhou 310052, China (e-mail: chengyu.wcy@alibaba-inc.com).

Corresponding authors: Chengyu Wang and Richang Hong.

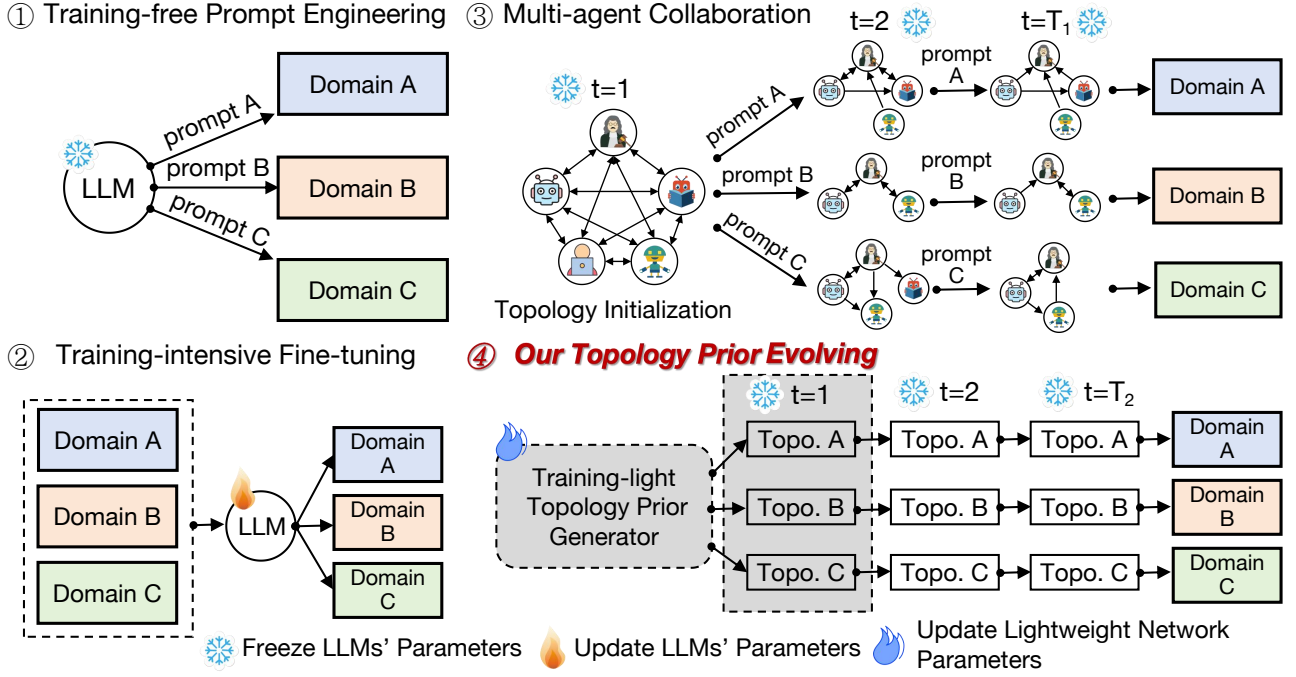


Fig. 1: Comparison of reasoning paradigms across domains. (1) **Training-free methods** rely on frozen LLMs and are ultimately limited by the intrinsic capability of the underlying model. (2) **Training-intensive methods** update model parameters, but may incur catastrophic forgetting and substantial computational overhead across domains. (3) **Multi-agent methods** dynamically construct collaboration topologies, yet repeatedly optimizing graphs from scratch for each query can incur high online token cost. Our method learns transferable topology priors that provide query-aware initialization for downstream topology evolution.

variational graph framework to capture reusable structural regularities in collaboration graphs across domains. Specifically, a variational encoder maps collaboration graphs and their associated queries into a latent space, while a conditional generator reconstructs query-conditioned initial topologies from the learned prior and the input query. This design enables TopoPrior to model collaboration structures that can be reused across related domains, rather than relearned independently for each query. (2) **Query-Conditioned Latent Adaptation.** While transferable priors can improve initialization efficiency, effective collaboration still requires sensitivity to query-relevant structural variation. We therefore introduce an adversarial latent adaptation module to reduce unnecessary domain discrepancy while preserving query-dependent structural characteristics. As a result, the generated topologies reflect both reusable collaboration regularities and task-specific specialization.

We evaluate TopoPrior on MMLU [2] and C-Eval [3] under multi-domain settings. Experimental results show that integrating TopoPrior with existing topology-evolution backbones improves downstream performance in most evaluated settings, reduces online inference-time token usage by up to **40.22%**, and introduces only **3.3M** additional trainable parameters for the topology-prior generator. These results suggest that transferable topology initialization can serve as an effective and lightweight mechanism for improving the efficiency of multi-agent LLM collaboration across domains.

The main contributions of this work are threefold:

- We formulate topology initialization for multi-agent LLM systems in multi-domain settings as a transferable topology

prior learning problem, in which reusable collaboration structures are learned offline and reused to improve downstream topology evolution.

- We propose TopoPrior, a lightweight framework that combines conditional variational topology prior learning with query-conditioned latent adaptation to generate informative initial collaboration graphs.
- We conduct experiments on MMLU and C-Eval with multiple topology-evolution backbones, showing that TopoPrior improves downstream reasoning performance and online inference token efficiency across the evaluated settings with modest parameter overhead.

II. RELATED WORK

A. Multi-Domain Reasoning with LLMs

Reasoning across multiple domains has become an increasingly important topic with the rise of LLMs [19]–[21]. Existing approaches mainly follow two paradigms. *Training-free methods* adapt frozen LLMs through in-context learning, prompt engineering, and chain-of-thought reasoning [6]–[8], [22]. They are easy to deploy and avoid parameter updates, but their performance is ultimately limited by the capability of the underlying model. *Training-intensive methods* adapt LLMs through supervised fine-tuning or parameter-efficient tuning such as LoRA [23]. Although effective for domain specialization, they may suffer from catastrophic forgetting and require additional storage and maintenance when many domains are involved [12], [13], [24], [25]. In contrast to prompt-level or parameter-level adaptation, our work studies a different axis of adaptation, namely structural adaptation at the

level of inter-agent communication. Specifically, rather than modifying prompts or updating LLM parameters, we improve multi-domain reasoning efficiency by learning reusable topology priors for multi-agent collaboration.

B. Topology Design in Multi-Agent LLM Systems

Multi-agent LLM systems improve reasoning by organizing agent interactions through communication topologies. Early methods mainly adopt predefined interaction patterns, including independent aggregation [26]–[28], chain-based communication [14], [29], [30], star-style coordination [31], [32], and tree-structured hierarchies [33]. While effective, these fixed designs are often inflexible when task requirements vary across inputs or domains.

More recent studies dynamically construct or optimize collaboration graphs. GPTSwarm [34] and DyLAN [15] adapt agent interactions through reinforcement learning or dynamic agent selection. Pruning-based methods remove redundant edges or agents to obtain task-adaptive sparse graphs [16], [35], while autoregressive approaches generate collaboration structures conditioned on the input query [17], [36]. Although these methods have shown strong reasoning performance, most of them construct or optimize topologies at query time and often search from scratch, incurring substantial online search cost and inference-time token overhead. Our work is complementary to this line of research: rather than replacing downstream topology evolution, we learn transferable topology priors that provide stronger initialization for it.

C. Transferable Structure Learning and Graph Initialization

Reusing structural knowledge across tasks or domains is important for efficient adaptation. Prior work in domain generalization and representation transfer suggests that shared latent structure can support transfer across heterogeneous settings [12], [37], [38]. In graph representation and generation learning, variational and conditional generative frameworks have been used to model reusable structural patterns from observed graphs [39], [40]. More broadly, warm-start initialization and structure reuse have long been recognized as practical strategies for reducing optimization cost in complex search problems [41], [42]. However, existing multi-agent LLM methods treat topology construction as a query-level search problem without learning transferable priors across domains, thus failing to amortize reusable collaboration patterns and keeping graph construction query-specific.

In contrast, our work introduces two key differences: (1) learning a *transferable topology prior* for graph initialization instead of searching from scratch per query; (2) combining prior learning with query-conditioned latent adaptation to preserve both cross-domain regularities and query-specific specialization. This design amortizes topology-search cost across domains and enhances downstream efficiency.

III. TOPOPRIOR: FRAMEWORK

In this section, we formalize the problem of transferable topology prior learning for multi-agent LLM collaboration in multi-domain settings and present the proposed TopoPrior framework. An overview of TopoPrior is shown in Fig. 2.

A. Problem Definition

Assume that we are given training data from K domains, denoted by $\mathcal{D} = \bigcup_{k=1}^K \mathcal{D}^k$, where $\mathcal{D}^k = \{(q_i^k, \mathbb{G}_i^k, y_i^k)\}_{i=1}^{M_k}$. Here, q_i^k is a query from the k -th domain, y_i^k is the corresponding ground-truth answer, and M_k is the number of samples in that domain. $\mathbb{G}_i^k = (\mathcal{V}_i^k, \mathcal{E}_i^k)$ denotes a reference collaboration graph produced by a strong topology-evolution method.¹ The reference graph is not assumed to be globally optimal; rather, it serves as an effective but imperfect source of structural supervision for learning transferable topology priors across domains.

Let $\mathcal{A}_i^k \in \mathbb{R}^{N \times N}$ denote the adjacency matrix derived from $\mathbb{G}_i^k$, where N is the size of the candidate role pool and serves as an upper bound on the number of generated agent nodes. The goal of TopoPrior is to learn a query-conditioned topology initializer that maps a new query q to an initial collaboration graph $\hat{\mathbb{G}}$, such that $\hat{\mathbb{G}}$ provides an informative starting point for downstream topology evolution. At inference time, the learned initializer is integrated with an existing topology-evolution backbone, which further refines $\hat{\mathbb{G}}$ according to its original task-specific optimization process. In our framework, topology-prior learning primarily uses query-graph pairs $(q_i^k, \mathbb{G}_i^k)$, while the task labels y_i^k are retained for reference-graph construction and downstream task evaluation.

B. Transferable Topology Prior Learning

The first component of TopoPrior aims to learn reusable topology priors that can initialize collaboration graphs across domains. Since queries from different domains may induce distinct yet structurally related collaboration patterns, we adopt a conditional variational graph framework [39] to encode reference graphs and queries into a shared latent space and reconstruct query-conditioned initial topologies from the resulting latent prior.

Formally, given a query-graph pair $(q, \mathbb{G})$, we model the conditional likelihood of the collaboration graph as

$$\log p_{\theta}(\mathbb{G} | q) \geq \mathbb{E}_{z \sim q_{\phi}(z | \mathbb{G}, q)} [\log p_{\theta}(\mathbb{G} | z, q)] - \text{KL}(q_{\phi}(z | \mathbb{G}, q) \| p'_{\theta}(z | q)), \quad (1)$$

where $q_{\phi}(z | \mathbb{G}, q)$ is the variational encoder, $p_{\theta}(\mathbb{G} | z, q)$ is the conditional graph generator, and $p'_{\theta}(z | q)$ is a query-conditioned prior. This formulation allows TopoPrior to capture reusable structural regularities from reference collaboration graphs while preserving query-specific adaptation through the conditional prior.

Variational Encoder. Given a query and its reference graph $(q, \mathbb{G})$, the encoder $q_{\phi}(z | \mathbb{G}, q)$ uses a Graph Convolutional Network (GCN) [43] to encode the collaboration graph²:

$$\mathbf{h}_v^{(l+1)} = f_{\sigma} \left(\tilde{\mathcal{D}}^{-\frac{1}{2}} \tilde{\mathcal{A}}_i^k \tilde{\mathcal{D}}^{-\frac{1}{2}} \mathbf{h}_v^{(l)} \mathbf{W}_{\text{ve}}^{(l)} \right), \quad (2)$$

where $\tilde{\mathcal{A}}_i^k = \mathcal{A}_i^k + I$ is the adjacency matrix with self-loops, $\tilde{\mathcal{D}}$ is the corresponding degree matrix, and $\mathbf{W}_{\text{ve}}^{(l)}$ are

¹In our implementation, we use AgentDropout [35] to construct reference collaboration graphs.

²All other alternative graph modeling methods can serve as replacements for this practice.

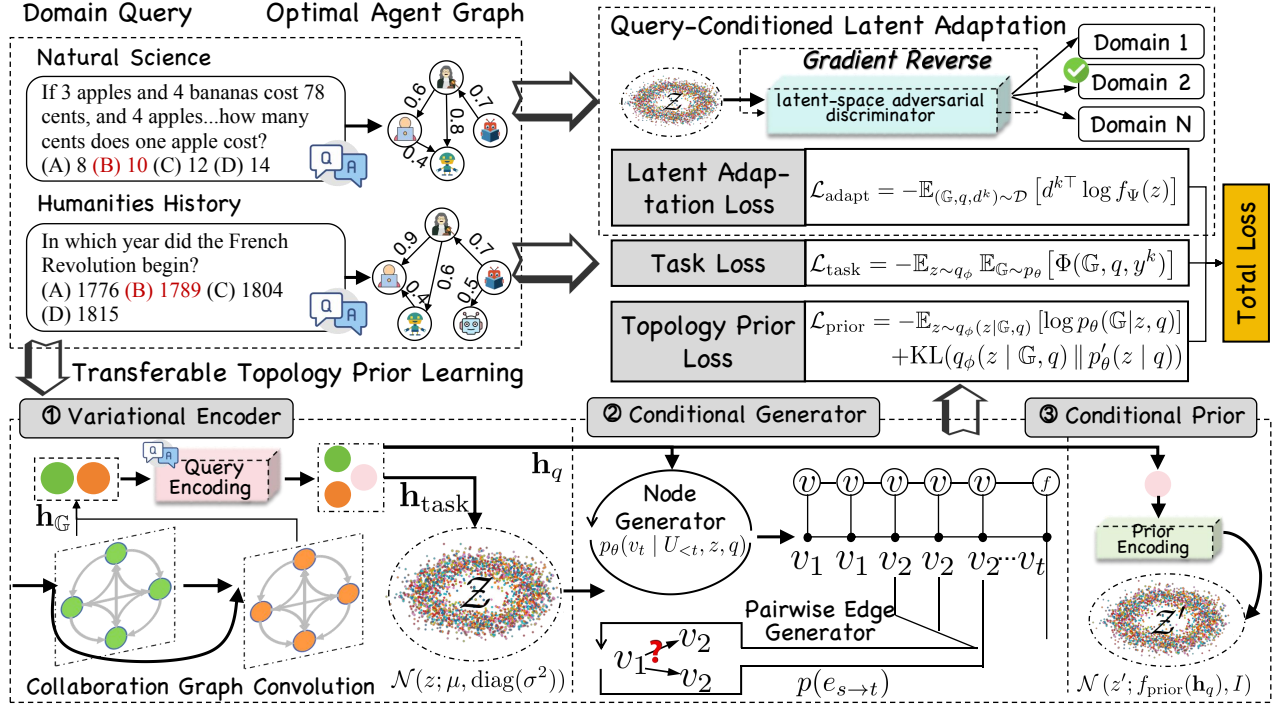


Fig. 2: Overview of TopoPrior. (1) *Transferable Topology Prior Learning* captures reusable collaboration patterns from multiple domains through a conditional variational graph framework. (2) *Query-Conditioned Latent Adaptation* improves cross-domain robustness by adversarially regularizing the latent space while retaining query-relevant structural information.

trainable weights. The query representation $\mathbf{h}_q$ and the initial agent-role representation $\mathbf{h}_v^{(0)}$ are obtained from a frozen text encoder such as BERT [44]. We then compute a graph-level representation by sum pooling over node embeddings:

$$\mathbf{h}_G = \sum_{n=1}^N \mathbf{h}_{v_n}. \quad (3)$$

The query and graph representations are fused as

$$\mathbf{h}_{\text{task}} = \text{MLP}_{\text{task}}(\mathbf{h}_q \parallel \mathbf{h}_G). \quad (4)$$

The latent variable z is sampled from the Gaussian posterior:

$$\boldsymbol{\mu} = \mathbf{W}_{\boldsymbol{\mu}} \mathbf{h}_{\text{task}} + \mathbf{b}_{\boldsymbol{\mu}}, \quad (5)$$

$$\log \sigma^2 = \mathbf{W}_{\sigma} \mathbf{h}_{\text{task}} + \mathbf{b}_{\sigma}, \quad (6)$$

$$q_{\phi}(z \mid \mathbb{G}, q) = \mathcal{N}(z; \boldsymbol{\mu}, \text{diag}(\sigma^2)), \quad (7)$$

where $z = \boldsymbol{\mu} + \sigma \odot \varepsilon$, $\varepsilon \sim \mathcal{N}(0, I)$, and $\sigma = \exp(\frac{1}{2} \log \sigma^2)$.

Conditional Generator. The conditional generator reconstructs an initial collaboration graph autoregressively:

$$p_{\theta}(\mathbb{G} \mid z, q) = \prod_{t=1}^T p_{\theta}(v_t \mid U_{<t}, z, q) \times \prod_{t=2}^T \prod_{s=1}^{t-1} p_{\theta}(e_{s \rightarrow t} \mid v_s, v_t, U_{<t}, z, q), \quad (8)$$

where $U_{<t}$ denotes the set of previously generated nodes, and T is the total number of generation steps. $p_{\theta}(e_{s \rightarrow t})$ represents the generation probability of edges between nodes. If it is greater than the threshold δ_e during inference, a newly generated edge will be added to $\mathcal{E}$. In our implementation,

node generation selects agent roles from a predefined role pool, and edge generation predicts directed communication links between generated nodes conditioned on the latent prior and the query representation. In practice, T is bounded by the candidate role pool, and unselected roles are omitted from the generated graph. This autoregressive design enables TopoPrior to produce sparse, query-adaptive initial collaboration graphs for downstream refinement.

Conditional Prior. The conditional prior $p'_{\theta}(z \mid q)$ provides a query-specific latent prior for topology generation:

$$p'_{\theta}(z \mid q) = \mathcal{N}(z; f_{\text{prior}}(\mathbf{h}_q), I), \quad (9)$$

where f_{prior} is a trainable MLP. Since reference graphs are unavailable at inference time, this conditional prior plays an important role in mapping unseen queries to useful regions of the topology latent space. The resulting topology-prior learning objective is

$$\mathcal{L}_{\text{prior}} = -\mathbb{E}_{z \sim q_{\phi}(z \mid \mathbb{G}, q)} [\log p_{\theta}(\mathbb{G} \mid z, q)] + \text{KL}(q_{\phi}(z \mid \mathbb{G}, q) \parallel p'_{\theta}(z \mid q)). \quad (10)$$

C. Query-Conditioned Latent Adaptation

While transferable topology priors can improve initialization efficiency, effective collaboration still requires sensitivity to query-relevant structural cues that may vary across domains. To this end, we introduce a domain-adversarial discriminator on the latent variable z to reduce unnecessary domain discrepancy during training. The discriminator is defined as

$$f_{\Psi}(z) = f_d(\mathbf{W}_d z + \mathbf{b}_d), \quad (11)$$

$$\mathcal{L}_{\text{adapt}} = -\mathbb{E}_{(\mathbb{G}, q, y^k) \sim \mathcal{D}} [y^k \log f_{\Psi}(z)], \quad (12)$$

Algorithm 1 TopoPrior Training Process

Require: Multi-domain training set $\mathcal{D} = \bigcup_{k=1}^K \mathcal{D}^k$, hyperparameter α and β , learning rate η , batch size B , number of epochs E

Ensure: Trained TopoPrior parameters θ, ϕ, Ψ

- 1: Initialize variational encoder $q_\phi(z|\mathbb{G}, q)$ with GCN layers
- 2: Initialize conditional generator $p_\theta(\mathbb{G}|z, q)$, prior network $p'_\theta(z|q)$, and domain discriminator $f_\Psi(z)$
- 3: **for** epoch = 1 to E **do**
- 4: **for** batch $(q_i, \mathbb{G}_i, d_i) \sim \mathcal{D}$ **do**
- 5: *Step 1: Encode graph and query*
- 6: Compute node representations $\mathbf{h}_v$ via GCN layers
- 7: Aggregate graph representation $\mathbf{h}_\mathbb{G} = \sum_v \mathbf{h}_v$
- 8: Encode query q_i into $\mathbf{h}_q$
- 9: Fuse query and graph features: $\mathbf{h}_{\text{task}} = \text{MLP}_{\text{task}}([\mathbf{h}_q \parallel \mathbf{h}_\mathbb{G}])$
- 10: Sample latent variable $z \sim q_\phi(z|\mathbb{G}, q)$ via reparameterization
- 11: *Step 2: Reconstruct graph*
- 12: Generate nodes and edges autoregressively via $p_\theta(\mathbb{G}|z, q)$
- 13: *Step 3: Compute losses*
- 14: $\mathcal{L}_{\text{recon}} = -\mathbb{E}_{z \sim q_\phi} [\log p_\theta(\mathbb{G}|z, q)]$
- 15: $\mathcal{L}_{\text{KL}} = \text{KL}(q_\phi(z|\mathbb{G}, q) \| p'_\theta(z|q))$
- 16: $\mathcal{L}_{\text{adapt}} = -\mathbb{E}[y \cdot \log f_\Psi(z)]$
- 17: *Step 4: Update parameters*
- 18: $\mathcal{L}_{\text{prior}} = \mathcal{L}_{\text{recon}} + \mathcal{L}_{\text{KL}}$
- 19: $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{prior}} + \alpha \mathcal{L}_{\text{task}} + \beta \mathcal{L}_{\text{adapt}}$
- 20: Update θ, ϕ, Ψ via $\nabla_{\theta, \phi, \Psi} \mathcal{L}_{\text{total}}$
- 21: **end for**
- 22: **end for**

where f_d is the softmax function and a gradient reversal layer (GRL) [45] is applied during training. Combined with query-conditioned prior learning and graph reconstruction, this adversarial objective reduces cross-domain discrepancy in the latent space, while the conditional prior and reconstruction objective help preserve query-dependent and task-relevant structural information for topology generation.

D. Training and Inference

The training objective of TopoPrior combines topology-prior learning and latent-space alignment:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{prior}} + \alpha \mathcal{L}_{\text{task}} + \beta \mathcal{L}_{\text{adapt}}, \quad (13)$$

where $\alpha, \beta \geq 0$ are hyperparameters, and $\mathcal{L}_{\text{task}}$ denotes the loss of the underlying task.

During training, TopoPrior learns transferable topology priors from reference graphs constructed on the training domains. During inference, given a new query q , we first sample $z \sim p'_\theta(z|q)$ and then generate an initial collaboration graph $\hat{\mathbb{G}} \sim p_\theta(\mathbb{G}|z, q)$. The generated graph is subsequently passed to a downstream topology-evolution backbone, which refines it according to its original task-specific optimization process. In this way, TopoPrior complements existing topology-evolution methods by providing lightweight and transferable graph initialization rather than replacing their task-specific search mechanisms. The training procedure for TopoPrior is summarized in Algorithm 1.

E. Analytical Perspective

We next provide an analytical perspective on the two design principles underlying TopoPrior. The goal of this subsection is not to establish end-to-end guarantees for LLM-based multi-agent topology evolution, but rather to formalize

the intuition behind latent alignment and topology initialization using standard analytical tools. Since both arguments are adapted from classical generalization and convergence analyses, we present concise proof sketches specialized to our setting. Detailed proofs are deferred to Appendix A.

1) *Cross-Domain Transfer via Latent Alignment:* Our framework uses adversarial latent alignment to reduce avoidable domain discrepancy across source domains, which may improve transfer to unseen domains. Let $P_1, \dots, P_K$ denote the marginal latent distributions of the K source domains induced by the encoder, and let P_t denote the marginal latent distribution of a target domain. For any hypothesis h in the class $\mathcal{H}$, define the expected error on domain d as

$$\epsilon_d(h) = \mathbb{E}_{z \sim P_d} [\ell_h(z)], \quad (14)$$

where ℓ_h is a bounded loss function.

We measure domain discrepancy in the latent space by the $\mathcal{H}\Delta\mathcal{H}$ -divergence:

$$d_{\mathcal{H}\Delta\mathcal{H}}(P, Q) = 2 \sup_{h, h' \in \mathcal{H}} \left| \Pr_{z \sim P} [h(z) \neq h'(z)] - \Pr_{z \sim Q} [h(z) \neq h'(z)] \right|. \quad (15)$$

By adapting standard multi-source domain adaptation analysis [46], [47] to the encoder-induced latent space, we obtain the following bound.

Theorem 1 (Multi-Source Domain Adaptation Bound in Latent Space). *Let $\alpha \in \Delta^{K-1}$ be convex weights over source domains, and let $P_\alpha = \sum_{k=1}^K \alpha_k P_k$. Then, for any $h \in \mathcal{H}$,*

$$\epsilon_t(h) \leq \sum_{k=1}^K \alpha_k \epsilon_k(h) + \frac{1}{2} d_{\mathcal{H}\Delta\mathcal{H}}(P_\alpha, P_t) + \lambda_{\alpha, t}^*, \quad (16)$$

where

$$\lambda_{\alpha, t}^* = \min_{h \in \mathcal{H}} \left(\sum_{k=1}^K \alpha_k \epsilon_k(h) + \epsilon_t(h) \right). \quad (17)$$

The result follows from applying standard multi-source domain adaptation arguments to the encoder-induced latent distributions and replacing input-space discrepancy with the corresponding $\mathcal{H}\Delta\mathcal{H}$ -divergence in latent space [46], [47]. Theorem 1 therefore provides a principled perspective on why reducing latent-space domain discrepancy may be beneficial for cross-domain transfer in TopoPrior. We emphasize that this result should be interpreted as motivation for latent-space regularization rather than as a task-specific guarantee for the full downstream system.

2) *Topology Initialization as Search Acceleration:* Our topology initializer is designed to provide stronger starting collaboration graphs, thereby reducing the number of refinement rounds required by downstream topology evolution. Let

$$J_\lambda(\mathbb{G}; q, y) = \text{Perf}(\mathbb{G}; q, y) - \lambda C(\mathbb{G}; q), \quad (18)$$

denote a utility function that combines task performance and communication cost, where $\lambda \geq 0$ controls the trade-off between the two terms. Define

$$U_t = \mathbb{E}[J_\lambda(\mathbb{G}_t; q, y)], \quad (19)$$

as the expected utility at evolution step t , starting from an initial graph $\mathbb{G}_0$.

To analyze the role of initialization, we adopt an abstract contraction-style assumption commonly used in convergence analysis of iterative optimization procedures [48]. Although downstream topology evolution in our setting is discrete and LLM-mediated, this assumption is used only as a simplified analytical model for studying the effect of initialization quality.

Assumption 1 (Linear Convergence). *There exists $\eta \in (0, 1]$ such that, for all t ,*

$$U^* - U_{t+1} \leq (1 - \eta)(U^* - U_t), \quad (20)$$

where U^* is the optimal achievable utility.

Under Assumption 1, we obtain the following round-complexity bound.

Theorem 2 (Rounds to ϵ -Suboptimality). *Under Assumption 1, to ensure $U^* - U_T \leq \epsilon$, it suffices that*

$$T \geq \frac{\log((U^* - U_0)/\epsilon)}{\log(1/(1 - \eta))}, \quad (21)$$

The bound is obtained by recursively unrolling Assumption 1, which gives $U^* - U_T \leq (1 - \eta)^T(U^* - U_0)$, and then solving for T .

Corollary 1 (Better Initialization Reduces Rounds). *If the prior-based initialization satisfies $U_0^{\text{prior}} > U_0^{\text{scratch}}$, then for any ϵ ,*

$$T_{\text{prior}}(\epsilon) - T_{\text{scratch}}(\epsilon) \leq \frac{\log\left(\frac{U^* - U_0^{\text{prior}}}{U^* - U_0^{\text{scratch}}}\right)}{\log(1/(1 - \eta))} < 0. \quad (22)$$

The corollary follows directly by comparing the bounds induced by U_0^{prior} and U_0^{scratch} in Theorem 2. It formalizes the intuition that better initialization can reduce the search cost of downstream topology evolution under the stated assumption. This interpretation is consistent with the lower token usage and fewer communication rounds observed in our experiments, although the result should be viewed as an analytical perspective rather than a literal guarantee for multi-agent LLM systems.

IV. EXPERIMENTS

In this section, we evaluate TopoPrior on multi-domain reasoning benchmarks and compare it with representative baselines, with a particular focus on topology-evolution methods. We report both downstream task performance and efficiency-related metrics, including online inference-time token usage and communication rounds. We further conduct ablation, sensitivity, and generalization analyses to examine the effectiveness of transferable topology prior learning under different settings.

A. Datasets

We evaluate TopoPrior on two widely used LLM benchmarks, MMLU [2] and C-Eval [3], under multi-domain settings. To construct domains for topology-prior learning, we group fine-grained benchmark subcategories into a smaller

TABLE I

Test-set statistics under the domain partitions used in our experiments. For training, we follow the official train/validation splits provided by MMLU and C-Eval.

Domain	Test Size	Domain	Test Size
MMLU			
Natural Sciences	6696	Ethics and Morality	558
Engineering and Technology	1116	Business and Management	1453
Social Sciences	2072	Humanities and History	1674
Law, Government, and Public Affairs	1825		
C-Eval			
Natural Sciences	2312	Vocational Qualifications and Professional Examinations	1581
Engineering and Technology	2424	Medicine and Life Sciences	1227
Social Sciences and Humanities	4618		
C-Eval Hard			
Mathematics	835	Chemistry	581
Physics	529		

number of major disciplinary categories according to semantic relatedness and the original benchmark taxonomy. The resulting test-set statistics are summarized in Table I.

MMLU. MMLU contains 57 subject-level tasks. We organize them into seven major domains according to their disciplinary themes: (1) *Natural Sciences*, (2) *Engineering and Technology*, (3) *Social Sciences*, (4) *Humanities and History*, (5) *Law, Government, and Public Affairs*, (6) *Ethics and Morality*, and (7) *Business and Management*. Together, these domains cover quantitative reasoning, technical problem solving, legal and policy analysis, ethical judgment, and knowledge-intensive social and historical understanding.

C-Eval. C-Eval contains 52 subject-level tasks designed to assess the general understanding capability of Chinese LLMs. We group these tasks into five major domains: (1) *Natural Sciences*, (2) *Engineering and Technology*, (3) *Social Sciences and Humanities*, (4) *Vocational Qualifications and Professional Examinations*, and (5) *Medicine and Life Sciences*. These categories cover both academic disciplines and professional qualification scenarios, providing a diverse testbed for multi-domain collaboration.

C-Eval Hard. To further evaluate robustness on challenging scientific reasoning tasks, we construct *C-Eval Hard* by extracting three natural-science subdomains from C-Eval: (1) *Mathematics*, (2) *Chemistry*, and (3) *Physics*. This benchmark focuses on difficult quantitative reasoning and serves as an additional evaluation setting for cross-domain transfer under domain variation.

B. Baselines

Training-free methods. We consider vanilla prompt engineering (PE) with Llama3-8B-Instruct [49], Qwen2.5-72B-Instruct [50], and DeepSeek-V3-671B-Instruct [51] as representative single-agent backbones. We further include Chain-of-Thought (CoT) [52] and Retrieval-Augmented Generation (RAG) [53] as stronger prompting-based baselines.

Training-intensive methods. We compare with representative multi-domain adaptation methods, including MoDULA [54],

TABLE II

Agent roles used in TopoPrior, aligned with the domain partitions of MMLU and C-Eval. Roles are selected dynamically according to the query content and task context.

Role Name	Domain	Role Description	Sub-domain
Natural Science Expert	Natural Sciences	Provides knowledge in physics, chemistry, biology, and medicine, and handles formal reasoning with domain-specific terminology.	Math, Physics, Chemistry, Biology
Engineering Specialist	Engineering & Technology	Solves queries in computer science, algorithms, security, and systems design using structured technical reasoning.	Computer Science, ML, Security
Social Scientist	Social Sciences	Analyzes economic, psychological, sociological, and political concepts, and performs causal and qualitative reasoning.	Economics, Psychology, Political Science
Humanities Scholar	Humanities & History	Interprets historical events, philosophical arguments, cultural contexts, and ethical frameworks.	History, Philosophy, World Religions
Legal Analyst	Law, Government, and Public Affairs	Interprets legal texts, statutes, treaties, and policy documents, and performs normative and juridical reasoning.	Jurisprudence, International Law
Ethics Consultant	Ethics & Morality	Evaluates moral dilemmas, ethical scenarios, and value-based judgments, especially in subjective contexts.	Moral Controversies, Ethical Scenarios
Business Strategist	Business & Management	Analyzes business ethics, accounting, marketing, and management strategies by combining normative and financial reasoning.	Business Ethics, Accounting, Marketing
Mathematical Expert	Mathematics	Solves mathematical queries, including discrete mathematics, probability, statistics, and algebra.	Discrete Math, Probability, Statistics
Chemistry Specialist	Chemistry	Answers chemistry questions at the high-school and college levels, and explains chemical reactions, properties, and theories.	General Chemistry, Organic Chemistry
Physics Specialist	Physics	Handles physics questions at the high-school and college levels, and applies principles from mechanics, electromagnetism, and related areas.	Mechanics, Electromagnetism
Medical Life Scientist	Medical	Provides knowledge in clinical medicine, veterinary science, agronomy, and plant sciences.	Clinical Medicine, Veterinary Science
Vocational Examiner	Vocational Qualifications	Answers certification-oriented questions in finance, taxation, civil service, and tourism.	CPA, Tax Agent, Civil Service
General Coordinator	Multi-domain	Orchestrates collaboration, aggregates outputs, and manages communication flow among specialized agents.	All domains (coordination role)

MoDE [24], and DES-MoE [25]. These methods improve domain specialization through parameter-efficient fine-tuning or mixture-of-experts architectures.

Training-light multi-agent methods. We evaluate TopoPrior on top of several topology-evolution backbones, including G-Designer [17], AgentPrune [16], ARG-Designer [36], and AgentDropout [35]. These methods dynamically optimize collaboration topologies through graph learning, pruning, autoregressive generation, or agent/edge dropout, respectively. Our method is complementary to these approaches, as it provides transferable graph initialization while leaving their downstream topology-refinement procedures unchanged.

C. Implementation Details

We implement TopoPrior in PyTorch and conduct experiments on two NVIDIA A800 GPUs. Unless otherwise specified, TopoPrior is trained separately on each benchmark under its corresponding multi-domain partition.

Backbones and encoders. Agent roles are instantiated using LLM backbones, including the locally deployed Llama3-8B-Instruct [49] and the online models Qwen2.5-72B-Instruct [50] and DeepSeek-V3-671B-Instruct [51]. The query representation $\mathbf{h}_q$ and the initial agent-role representation $\mathbf{h}_v^{(0)}$ are encoded using a frozen BERT encoder [44]. Unless otherwise stated, gold domain labels are used only for benchmark partitioning and analysis and are not provided to TopoPrior at inference time.

TopoPrior architecture. The variational encoder is implemented as a two-layer GCN [43] with hidden size $d_{\mathbf{h}_v} = 256$, latent dimension $d_z = 128$, and ReLU activation [55]. Linear projections are implemented with two-layer MLPs. For node and edge history encoding, we use a two-layer GRU [56]. The latent-space discriminator is implemented as a two-layer MLP with a softmax output layer. The edge-generation threshold is set to $\delta_e = 0.5$, and the gradient reversal layer uses a coefficient of -0.1 .

Training protocol. We optimize the model with Adam using a learning rate of 2×10^{-4} and a batch size of 32. The alignment coefficients are set to $\alpha = 0.5$, $\beta = 0.5$, and the topology-prior

generator is trained for five epochs. Reference collaboration graphs are constructed offline on the training split using AgentDropout [35]. During evaluation, the generated initial graph is passed to the corresponding downstream topology-evolution backbone for task-specific refinement. For fair comparison, integrating TopoPrior changes only the initialization stage of each backbone while keeping its downstream refinement procedure and search budget unchanged, unless otherwise specified.

Role pool and graph construction. The conditional generator $p_\theta(\mathbb{G} \mid z, q)$ autoregressively selects agent nodes from an extensible role pool. Table II summarizes the agent roles associated with the major domains in MMLU and C-Eval. For natural-science tasks, we include both a generic *Natural Science Expert* and specialized roles (e.g., *Mathematical Expert*, *Chemistry Specialist*, and *Physics Specialist*) to better support challenging subdomains. For efficient offline supervision construction, we use AgentDropout [35] to obtain reference communication topologies, as it is substantially faster than more iterative topology-construction methods on large training sets. Each agent node $v_{i,j}^k = \{\text{LLM}, \text{Role}, \text{State}, \text{Plugins}\}$ encodes the underlying language model, assigned role, current state, and any attached tools or plugins [16].

Evaluation protocol. Unless otherwise specified, we report classification accuracy (Acc.) on domain-specific test splits. For efficiency analysis, token consumption is measured as the total number of LLM input and output tokens incurred during online inference, including inter-agent communication and final answer generation, but excluding the offline cost of reference-graph construction and topology-prior training. We report these metrics as online inference efficiency and discuss offline supervision and training cost separately when relevant. For ARG-Designer [36], which generates graph topologies autoregressively from scratch, we incorporate the latent representation from TopoPrior into its initial query encoding (i.e., $\mathbf{f}_Q$ in its Eq. (6)) as the initialization signal.

D. Main Results

Table III and Table IV summarize the main results on MMLU and C-Eval under three LLM backbones. Overall,

TABLE III

Results on MMLU across domains with different LLM backbones. “GD”, “AP”, “ARG”, and “AD” denote G-Designer, AgentPrune, ARG-Designer, and AgentDropout, respectively. Best results are in bold and second-best results are underlined.

Domain Models	→ ↓	Natural Sciences	Engineering Technology	Social Sciences	Humanities History	Law, Government Public Affairs	Ethics Morality	Business Management	Average
Base model: Llama3-8B-Instruct									
PE		57.84	58.89	56.61	60.05	49.84	45.72	59.40	55.05
CoT		58.97 (↑1.13)	61.51 (↑2.62)	59.02 (↑2.41)	62.48 (↑2.43)	50.65 (↑0.81)	46.76 (↑1.04)	60.68 (↑1.28)	57.15
RAG		61.65 (↑3.81)	62.39 (↑3.50)	62.95 (↑6.34)	65.37 (↑5.32)	58.03 (↑8.19)	54.50 (↑8.78)	62.83 (↑3.43)	61.10
MoDULA		56.89 (↓0.95)	58.72 (↓0.17)	57.95 (↑1.34)	61.22 (↑1.17)	51.60 (↑1.76)	48.42 (↑2.70)	60.53 (↑1.13)	56.47
MoDE		59.72 (↑1.88)	63.05 (↑4.16)	62.45 (↑5.84)	64.69 (↑4.64)	56.79 (↑6.95)	54.11 (↑8.39)	61.06 (↑1.66)	60.12
DES-MoE		58.88 (↑1.04)	62.34 (↑3.45)	60.70 (↑4.09)	62.63 (↑2.58)	54.84 (↑5.00)	52.98 (↑7.26)	61.33 (↑1.93)	59.10
G-Designer		64.82 (↑6.98)	66.57 (↑7.68)	63.91 (↑7.30)	67.70 (↑7.65)	61.58 (↑11.74)	59.13 (↑13.41)	65.44 (↑6.04)	64.16
AgentPrune		63.99 (↑6.15)	66.85 (↑7.96)	64.73 (↑8.12)	66.98 (↑6.93)	63.07 (↑13.23)	58.51 (↑12.79)	64.12 (↑4.72)	64.04
ARG-Designer		65.48 (↑7.64)	68.01 (↑9.12)	64.95 (↑8.34)	69.54 (↑9.49)	63.86 (↑14.02)	61.32 (↑15.60)	65.87 (↑6.47)	65.58
AgentDropout		64.30 (↑6.46)	69.72 (↑10.83)	63.14 (↑6.53)	68.86 (↑8.81)	65.47 (↑15.63)	62.05 (↑16.33)	66.53 (↑7.13)	65.72
TopoPrior+GD		66.32 (↑8.48)	67.80 (↑8.91)	65.42 (↑8.81)	70.41 (↑10.36)	63.96 (↑14.12)	60.85 (↑15.13)	67.18 (↑7.78)	65.99
TopoPrior+AP		65.34 (↑9.86)	68.75 (↑9.86)	65.19 (↑8.58)	69.16 (↑9.11)	63.93 (↑14.09)	61.80 (↑16.08)	67.62 (↑8.22)	65.97
TopoPrior+ARG		68.91 (↑11.07)	71.57 (↑12.68)	68.13 (↑11.52)	72.74 (↑12.69)	<u>66.15</u> (↑16.31)	64.08 (↑18.36)	<u>68.14</u> (↑8.74)	68.53
TopoPrior+AD		65.70 (↑7.86)	<u>69.95</u> (↑11.06)	<u>65.47</u> (↑8.86)	70.36 (↑10.31)	66.88 (↑17.04)	<u>63.75</u> (↑18.03)	69.05 (↑9.65)	<u>67.31</u>
Base model: DeepSeek-V3-671B-Instruct									
PE		84.96	85.23	84.78	86.41	79.63	78.92	85.12	84.47
CoT		84.60 (↓0.36)	84.85 (↓0.38)	85.32 (↑0.54)	86.87 (↑0.46)	80.19 (↑0.56)	79.52 (↑0.60)	85.57 (↑0.45)	84.90
RAG		85.92 (↑0.96)	86.17 (↑0.94)	85.84 (↑1.06)	87.63 (↑1.22)	82.45 (↑2.82)	81.78 (↑2.86)	86.05 (↑0.93)	85.38
G-Designer		89.23 (↑4.27)	89.67 (↑4.44)	88.91 (↑4.13)	91.08 (↑4.67)	84.12 (↑4.49)	83.46 (↑4.54)	89.35 (↑4.23)	88.52
AgentPrune		89.84 (↑4.88)	90.03 (↑4.80)	89.52 (↑4.74)	91.47 (↑5.06)	84.73 (↑5.10)	83.98 (↑5.06)	89.76 (↑4.64)	89.19
ARG-Designer		91.15 (↑6.19)	91.80 (↑6.57)	91.07 (↑6.29)	93.69 (↑7.28)	86.28 (↑6.65)	85.52 (↑6.60)	91.48 (↑6.36)	90.14
AgentDropout		89.97 (↑5.01)	90.50 (↑5.27)	90.11 (↑5.33)	91.93 (↑5.52)	85.29 (↑5.66)	84.89 (↑5.97)	90.25 (↑5.13)	88.99
TopoPrior+GD		90.12 (↑5.16)	90.46 (↑5.23)	89.82 (↑5.04)	92.37 (↑5.96)	85.43 (↑5.80)	84.71 (↑5.79)	90.28 (↑5.16)	89.89
TopoPrior+AP		90.75 (↑5.79)	90.90 (↑5.67)	90.41 (↑5.63)	92.89 (↑6.48)	86.01 (↑6.38)	85.31 (↑6.39)	90.65 (↑5.53)	90.41
TopoPrior+ARG		92.87 (↑7.91)	93.14 (↑7.91)	92.55 (↑7.77)	95.12 (↑8.71)	87.83 (↑8.20)	87.08 (↑8.16)	92.79 (↑7.67)	92.03
TopoPrior+AD		91.78 (↑6.82)	92.09 (↑6.86)	91.33 (↑6.55)	93.95 (↑7.54)	86.95 (↑7.32)	86.51 (↑7.59)	91.74 (↑6.62)	90.62
Base model: Qwen2.5-72B-Instruct									
PE		84.17	84.44	83.86	85.53	79.84	79.19	83.92	83.26
CoT		85.02 (↑0.85)	84.40 (↓0.04)	84.79 (↑0.93)	86.71 (↑1.18)	81.17 (↑1.33)	80.42 (↑1.23)	85.07 (↑1.15)	84.41
RAG		85.71 (↑1.54)	86.02 (↑1.58)	85.48 (↑1.62)	87.43 (↑1.90)	82.33 (↑2.49)	81.58 (↑2.39)	85.73 (↑1.81)	85.56
G-Designer		87.42 (↑3.25)	87.55 (↑3.11)	87.20 (↑3.34)	88.97 (↑3.44)	83.19 (↑3.35)	82.58 (↑3.39)	87.43 (↑3.51)	86.33
AgentPrune		86.67 (↑2.50)	86.98 (↑2.54)	86.44 (↑2.58)	88.42 (↑2.89)	82.56 (↑2.72)	81.81 (↑2.62)	86.69 (↑2.77)	85.71
ARG-Designer		88.50 (↑4.33)	88.87 (↑4.43)	88.41 (↑4.55)	90.40 (↑4.87)	84.49 (↑4.65)	83.55 (↑4.36)	88.69 (↑4.77)	87.56
AgentDropout		87.63 (↑3.46)	87.79 (↑3.35)	87.40 (↑3.54)	89.32 (↑3.79)	83.59 (↑3.75)	82.70 (↑3.51)	87.74 (↑3.82)	86.60
TopoPrior+GD		88.31 (↑4.14)	88.44 (↑4.00)	88.13 (↑4.27)	90.01 (↑4.48)	84.30 (↑4.46)	83.29 (↑4.10)	88.32 (↑4.40)	87.26
TopoPrior+AP		87.74 (↑3.57)	87.61 (↑3.27)	87.49 (↑3.63)	89.33 (↑3.80)	83.58 (↑3.74)	82.73 (↑3.54)	87.64 (↑3.72)	86.59
TopoPrior+ARG		90.87 (↑6.70)	91.18 (↑6.74)	90.64 (↑6.78)	92.62 (↑7.09)	86.76 (↑6.92)	86.01 (↑6.82)	90.89 (↑6.97)	89.56
TopoPrior+AD		89.48 (↑5.31)	89.62 (↑5.18)	89.35 (↑5.49)	91.26 (↑5.73)	85.45 (↑5.61)	84.75 (↑5.56)	89.89 (↑5.97)	88.54

dynamic topology-evolution methods consistently outperform single-agent baselines in most evaluated settings, and equipping them with TopoPrior further improves performance in most cases. These results suggest that transferable topology priors can provide effective initialization for downstream multi-agent collaboration in multi-domain reasoning.

Compared with training-free methods, RAG is the strongest single-agent baseline in most settings, especially on knowledge-intensive domains such as *Law, Government, and Public Affairs, Ethics and Morality, and Social Sciences*. Training-intensive methods exhibit more mixed behavior across heterogeneous domains and are generally weaker than dynamic multi-agent approaches in our evaluation. By contrast, topology-evolution methods provide the strongest baseline results overall, which supports the value of role specialization and structured communication for multi-domain

reasoning.

Built on top of these topology-evolution backbones, TopoPrior yields consistent gains across benchmarks and model scales. For example, on MMLU with Llama3-8B-Instruct, TopoPrior+ARG improves the average accuracy from **65.58** to **68.53** (+2.95 points), and TopoPrior+AD improves AgentDropout from **65.72** to **67.31** (+1.59 points). On C-Eval with the same backbone, TopoPrior+ARG improves ARG-Designer from **54.25** to **57.04** (+2.79 points), while TopoPrior+AD improves AgentDropout from **53.84** to **56.01** (+2.17 points). Similar trends are observed on larger backbones, including DeepSeek-V3-671B-Instruct and Qwen2.5-72B-Instruct.

The improvements are particularly visible in reasoning-intensive and knowledge-intensive categories. On MMLU with Llama3-8B-Instruct, TopoPrior+ARG achieves the

TABLE IV

Results on C-Eval and C-Eval Hard across domains with LLM backbones. “GD”, “AP”, “ARG”, and “AD” denote G-Designer, AgentPrune, ARG-Designer, and AgentDropout. Best results are in bold and second-best results are underlined.

Domain Models	→ ↓	Natural Sciences	Engineering Technology	Social Sciences Humanities	Vocational Qualif. Prof. Exam.	Medicine Life Sci.	Math	C-Eval Hard Chemistry	Physics	Average
Base model: Llama3-8B-Instruct										
PE		54.73	53.64	55.02	48.69	46.25	41.83	44.06	40.12	48.04
CoT		55.85(↑1.12)	54.47(↑0.83)	55.48(↑0.46)	50.36(↑1.67)	48.07(↑1.82)	42.99(↑1.16)	45.80(↑1.74)	41.84(↑1.72)	49.36
RAG		57.45(↑2.72)	56.59(↑2.95)	56.98(↑1.96)	51.75(↑3.06)	50.58(↑4.33)	44.83(↑3.00)	47.06(↑3.00)	44.12(↑4.00)	51.17
MoDULA		54.37(↓0.36)	52.95(↓0.69)	55.84(↑0.82)	49.72(↑1.03)	47.55(↑1.30)	41.80(↓0.03)	45.62(↑1.56)	41.23(↑1.11)	48.64
MoDE		54.61(↑0.12)	53.97(↑0.33)	56.14(↑1.12)	50.80(↑2.11)	49.25(↑3.00)	42.77(↑0.94)	44.02(↓0.04)	42.52(↑2.40)	49.26
DES-MoE		55.15(↑0.42)	53.42(↓0.22)	56.50(↑1.48)	51.31(↑2.62)	48.84(↑2.59)	43.96(↑2.13)	43.58(↓0.48)	41.96(↑1.84)	49.34
G-Designer		59.60(↑4.87)	57.86(↑4.22)	57.15(↑2.13)	54.08(↑5.39)	52.63(↑6.38)	45.91(↑4.08)	50.11(↑6.05)	46.32(↑6.20)	52.96
AgentPrune		59.48(↑4.75)	58.16(↑4.52)	56.93(↑1.91)	54.50(↑5.81)	52.09(↑5.84)	45.24(↑3.41)	50.85(↑6.79)	45.88(↑5.76)	52.89
ARG-Designer		60.82(↑6.09)	59.52(↑5.88)	57.11(↑2.09)	56.27(↑7.58)	53.85(↑7.60)	46.73(↑4.90)	52.59(↑8.53)	47.14(↑7.02)	54.25
AgentDropout		60.40(↑5.67)	58.86(↑5.22)	57.10(↑2.08)	55.79(↑7.10)	53.54(↑7.29)	45.98(↑4.15)	52.10(↑8.04)	46.96(↑6.84)	53.84
TopoPrior+GD		61.83(↑7.10)	58.72(↑5.08)	59.05(↑4.03)	56.51(↑7.82)	54.30(↑8.05)	46.95(↑5.12)	52.63(↑8.57)	48.41(↑8.29)	54.80
TopoPrior+AP		61.16(↑6.43)	60.35(↑6.71)	58.84(↑3.82)	56.07(↑7.38)	53.98(↑7.73)	47.72(↑5.89)	52.93(↑8.87)	48.22(↑8.10)	54.91
TopoPrior+ARG		63.54 (↑8.81)	62.60 (↑8.96)	60.21 (↑5.19)	58.15 (↑9.46)	55.82 (↑9.57)	49.33 (↑7.50)	55.76 (↑11.70)	50.89 (↑10.77)	57.04
TopoPrior+AD		62.80(↑8.07)	61.42(↑7.78)	58.96(↑3.94)	57.40(↑8.71)	55.36(↑9.11)	48.44(↑6.61)	53.86(↑9.80)	49.87(↑9.75)	56.01
Base model: DeepSeek-V3-671B-Instruct										
PE		86.88	84.75	87.94	79.76	77.39	71.21	74.48	69.64	79.01
CoT		86.63(↓0.25)	85.62(↑0.87)	88.64(↑0.70)	81.75(↑1.99)	78.35(↑0.96)	72.78(↑1.57)	75.62(↑1.14)	70.81(↑1.17)	80.03
RAG		88.82(↑1.94)	86.88(↑2.13)	89.70(↑1.76)	81.92(↑2.16)	79.54(↑2.15)	73.39(↑2.18)	76.85(↑2.37)	72.03(↑2.39)	81.14
G-Designer		90.35(↑3.47)	88.52(↑3.77)	91.40(↑3.46)	83.43(↑3.67)	80.82(↑3.43)	75.28(↑4.07)	78.93(↑4.45)	74.14(↑4.50)	82.86
AgentPrune		90.62(↑3.74)	88.84(↑4.09)	90.98(↑3.04)	84.11(↑4.35)	81.08(↑3.69)	74.45(↑3.24)	81.15(↑6.67)	75.32(↑5.68)	83.32
ARG-Designer		91.49(↑4.61)	89.87(↑5.12)	91.90(↑3.96)	85.83(↑6.07)	82.76(↑5.37)	76.50(↑5.29)	81.97(↑7.49)	76.06(↑6.42)	84.42
AgentDropout		90.83(↑3.95)	88.15(↑3.40)	91.22(↑3.28)	83.64(↑3.88)	81.66(↑4.27)	76.17(↑4.96)	80.94(↑6.46)	75.80(↑6.16)	83.55
TopoPrior+GD		91.80(↑4.92)	89.67(↑4.92)	92.52(↑4.58)	85.33(↑5.57)	82.95(↑5.56)	76.84(↑5.63)	80.50(↑6.02)	76.69(↑7.05)	84.54
TopoPrior+AP		91.19(↑4.31)	<u>90.51</u> (↑5.76)	92.27(↑4.33)	85.30(↑5.54)	82.78(↑5.39)	77.03(↑5.82)	<u>82.41</u> (↑7.93)	<u>76.98</u> (↑7.34)	84.81
TopoPrior+ARG		93.02 (↑6.14)	92.46 (↑7.71)	93.08 (↑5.14)	86.68 (↑6.92)	84.83 (↑7.44)	78.31 (↑7.10)	83.94 (↑9.46)	78.06 (↑8.42)	86.30
TopoPrior+AD		91.75(↑4.87)	90.32(↑5.57)	91.79(↑3.85)	<u>85.96</u> (↑5.00)	<u>83.03</u> (↑5.64)	<u>77.14</u> (↑5.93)	81.95(↑7.47)	76.40(↑6.76)	84.79
Base model: Qwen2.5-72B-Instruct										
PE		84.42	82.55	85.29	78.30	76.57	70.55	74.02	68.18	77.49
CoT		84.53(↑0.11)	82.10(↓0.45)	85.27(↓0.02)	80.19(↑1.89)	78.24(↑1.67)	71.79(↑1.24)	75.33(↑1.31)	69.85(↑1.67)	78.41
RAG		85.40(↑0.98)	84.06(↑1.51)	86.55(↑1.26)	82.12(↑3.82)	79.69(↑3.12)	73.40(↑2.85)	76.73(↑2.71)	71.31(↑3.13)	79.91
G-Designer		86.58(↑2.16)	85.21(↑2.66)	88.07(↑2.78)	84.29(↑5.99)	80.28(↑3.71)	74.99(↑4.44)	78.87(↑4.85)	73.96(↑5.78)	81.53
AgentPrune		86.03(↑1.61)	86.15(↑3.60)	88.22(↑2.93)	85.93(↑7.63)	80.79(↑4.22)	75.47(↑4.92)	79.34(↑5.32)	74.84(↑6.66)	82.10
ARG-Designer		87.63(↑3.21)	87.72(↑5.17)	89.16(↑3.87)	86.57(↑8.27)	81.85(↑5.28)	76.74(↑6.19)	81.41(↑7.39)	76.45(↑8.27)	82.19
AgentDropout		86.57(↑2.15)	85.70(↑3.15)	87.96(↑2.67)	85.52(↑7.22)	80.44(↑3.87)	75.30(↑4.75)	79.39(↑5.37)	74.68(↑6.50)	81.95
TopoPrior+GD		87.65(↑3.23)	87.10(↑4.55)	88.74(↑3.45)	86.65(↑8.35)	82.11(↑5.54)	77.16(↑6.61)	<u>81.52</u> (↑7.50)	76.51(↑8.33)	82.18
TopoPrior+AP		88.67(↑4.25)	86.84(↑4.29)	89.73(↑4.44)	86.92(↑8.62)	81.90(↑5.33)	77.65(↑7.10)	80.44(↑6.42)	76.69(↑8.51)	82.36
TopoPrior+ARG		90.16 (↑5.74)	89.25 (↑6.70)	91.21 (↑5.92)	88.07 (↑9.77)	83.32 (↑6.75)	79.09 (↑8.54)	83.50 (↑9.48)	79.91 (↑11.73)	85.56
TopoPrior+AD		88.59(↑4.17)	87.68(↑5.13)	88.64(↑3.35)	86.53(↑8.23)	<u>82.81</u> (↑6.24)	77.55(↑7.00)	81.36(↑7.34)	<u>77.14</u> (↑8.96)	83.79

best performance in *Ethics and Morality* (**64.08**) and ranks among the strongest methods in several other domains, while TopoPrior+AD performs best on *Law, Government, and Public Affairs* (**66.88**) and *Business and Management* (**69.05**). On C-Eval and C-Eval Hard, the gains are also evident on challenging scientific reasoning tasks. For instance, under Llama3-8B-Instruct, TopoPrior+ARG reaches **49.33**, **55.76**, and **50.89** on Mathematics, Chemistry, and Physics, respectively, outperforming the corresponding backbone without topology-prior initialization.

Among the evaluated backbones, ARG-Designer appears to benefit the most from TopoPrior. This trend is observed across both benchmarks and all three LLM backbones, suggesting that autoregressive graph construction may be particularly sensitive to initialization quality. More broadly, the fact that TopoPrior improves several heterogeneous backbones,

rather than only the method used to construct the reference graphs, suggests that it captures reusable collaboration regularities beyond a teacher-specific search pattern.

E. Ablation Study

Table V reports ablation results on three representative MMLU domains under two LLM backbones. Using TopoPrior+ARG as the full model, we examine the contributions of transferable topology-prior learning, latent adaptation, and the query-conditioned prior. Removing $\mathcal{L}_{\text{prior}}$ causes the largest performance drop (up to 4.27 points on Llama3-8B and 5.72 points on Qwen2.5-72B), indicating that topology-prior learning is the central component of TopoPrior. Without this objective, the model is less able to capture reusable structural regularities, leading to weaker graph initialization. Removing $\mathcal{L}_{\text{adapt}}$ also degrades performance on

TABLE V

Ablation results of TopoPrior on three representative MMLU domains using ARG-Designer.

Domain→ Model↓	Natural Sciences	Law, Gov. Public Affairs	Ethics Morality
Base model: Llama3-8B-Instruct			
TopoPrior+ARG	68.91	66.15	64.08
w/o $\mathcal{L}_{\text{prior}}$	64.71	62.92	59.81
w/o $\mathcal{L}_{\text{adapt}}$	66.14	64.68	62.50
w/o f_{prior}	65.31	63.70	60.25
Base model: Qwen2.5-72B-Instruct			
TopoPrior+ARG	90.87	86.76	86.01
w/o $\mathcal{L}_{\text{prior}}$	85.15	83.92	83.26
w/o $\mathcal{L}_{\text{adapt}}$	87.83	85.57	85.13
w/o f_{prior}	86.42	85.06	84.55

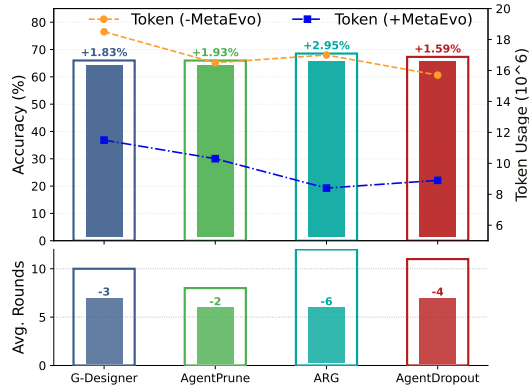


Fig. 3: Accuracy gain, inference-time token reduction, and communication-round reduction of TopoPrior when combined with different topology-evolution backbones on MMLU using Llama3-8B-Instruct.

both backbones, although less severely than removing $\mathcal{L}_{\text{prior}}$. This suggests that latent-space alignment improves the robustness of the learned topology prior across domains by reducing domain discrepancy while still preserving useful query-dependent structural cues. Removing the query-conditioned prior f_{prior} further reduces performance on both backbones. This result suggests that the learned prior remains important at inference time: without it, the generator loses an informative query-specific initialization signal and becomes less effective at constructing initial collaboration graphs for downstream refinement. Overall, the three components are complementary. The topology-prior objective contributes the largest gain, while latent adaptation and the query-conditioned prior provide additional improvements for more stable topology initialization. Their relative ordering is similar across the two backbones, suggesting that these effects are reasonably stable across model scales.

F. Efficiency Analysis

a) *Efficiency–Performance Trade-off.*: We assess the trade-off among performance gain, online token efficiency, and communication-round reduction by equipping TopoPrior with different topology-evolution backbones on MMLU using Llama3-8B-Instruct. Following the protocol described in

TABLE VI

Comparison of average accuracy and trainable parameters on MMLU with Llama3-8B-Instruct. Relative parameter increase (Δ) is computed w.r.t. the 8B backbone.

Method	Acc. (%)	Trainable Parameters (M)	Δ (%)
MoDULA	56.47	1,775	22.19
MoDE	60.12	2,376	29.70
DES-MoE	59.10	1,585	19.81
TopoPrior+ARG	68.53	3.3+3.8=7.1	0.09

TABLE VII

Performance of TopoPrior under weak and strong topology supervision.

Setting	Accuracy (%)
Weak Teacher (50% convergence)	61.24
TopoPrior + Weak Teacher	64.33
Strong Teacher (full convergence)	65.58
TopoPrior + Strong Teacher	68.53

Section IV-C, token consumption is measured over online inference only. We also compare training-intensive baselines to assess adaptation quality relative to trainable parameter cost.

Figure 3 shows that TopoPrior improves all evaluated topology-evolution methods while reducing online communication cost. The outer box labeled “Acc.” and the inner box labeled “Rounds” indicate changes in accuracy and communication rounds, respectively. Among the evaluated methods, ARG-Designer appears to benefit the most, achieving the largest average accuracy gain (+2.95 points) together with the largest reduction in communication rounds (6 rounds on average). This observation is consistent with the main results and suggests that autoregressive graph construction may be particularly sensitive to initialization quality. By contrast, pruning-based methods such as AgentPrune benefit less from improved initialization, possibly because their downstream refinement process retains more redundant agents and edges.

Table VI further summarizes adaptation performance and parameter cost. TopoPrior+ARG achieves the best average accuracy (68.53%) while introducing only 3.3M additional parameters in TopoPrior itself; the total trainable parameter count of TopoPrior+ARG is 7.1M, including 3.8M parameters from the original ARG-Designer. This increase is small relative to the 8B backbone. By contrast, training-intensive baselines require substantially more trainable parameters while achieving lower average accuracy on this benchmark. These results suggest that TopoPrior provides a favorable trade-off between adaptation quality and parameter efficiency under the evaluated setting. We note, however, that these efficiency results characterize online inference savings and parameter overhead, rather than the full amortized cost including offline reference-graph construction and prior training.

b) *Weak Topology Supervision.*: We further examine whether TopoPrior remains effective under degraded and simplified supervision. Table VII compares strong teacher graphs obtained from fully converged AgentDropout with weak teacher graphs obtained by stopping AgentDropout after 50% of its convergence rounds. Even under weak teacher supervision, TopoPrior reaches 64.33% accuracy, improving

TABLE VIII

Effect of different supervision sources on model performance and inference-time token efficiency. Token reduction is measured against the ARG-Designer baseline.

Supervision Source	Accuracy (%)	Token Reduction (%)
ARG-Designer	65.58	—
+ <i>Full</i>	68.53	40.2
+ <i>Cheap-early</i>	64.33	32.7
+ <i>Static-template</i>	62.14	18.3
+ <i>Random</i>	60.25	20.5

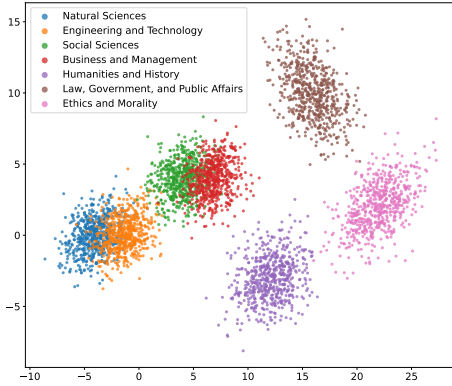


Fig. 4: t-SNE visualization of the encoder-induced latent space across the seven MMLU domains.

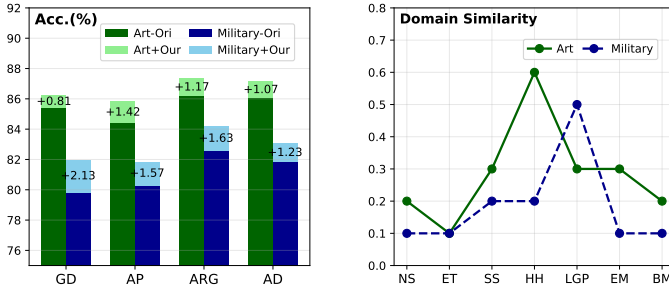


Fig. 5: Out-of-domain generalization on unseen domains. “GD”, “AP”, “ARG”, and “AD” denote G-Designer, Agent-Prune, ARG-Designer, and AgentDropout, respectively.

over the weak teacher itself by 3.09 points and remaining only 1.25 points below the fully supervised setting. This result suggests that TopoPrior can still extract useful collaboration patterns from imperfect topology supervision.

We further compare alternative supervision sources in Table VIII, including fully converged graphs (*Full*), partially converged graphs (*Cheap-early*), domain-heuristic static templates (*Static-template*), and random graphs. All variants are used to train TopoPrior, and the resulting generator is evaluated by initializing ARG-Designer. The results show that TopoPrior remains effective even when supervision is simplified: partially converged graphs already provide clear improvements, while static templates still yield moderate gains. By contrast, random graphs do not produce competitive results, suggesting that TopoPrior learns meaningful collaboration patterns rather than merely fitting superficial graph statistics.

G. Representation and Generalization Analysis

a) *Multi-Domain Latent Structure.*: To assess whether TopoPrior learns latent representations that are both reusable and domain-sensitive, we visualize the latent variable z for all seven major MMLU domains. For each domain, we sample 400 examples, encode them into the latent space using the learned encoder, and project the resulting 128-dimensional representations to two dimensions using PCA followed by t-SNE [57]. Figure 4 shows that the seven domains form several coherent clusters in the latent space. In particular, *Natural Sciences* and *Engineering & Technology* exhibit partial overlap, likely due to shared quantitative and formal reasoning patterns, while *Social Sciences* and *Business & Management* also overlap to some extent because of related economic and decision-oriented concepts. Other domains remain relatively distinct. Although this visualization is qualitative, it is consistent with the intended behavior of TopoPrior: the latent space captures reusable structural regularities while retaining domain-related variation that may be useful for topology initialization.

b) *Out-of-Domain Generalization.*: To evaluate generalization beyond the training domains, we conduct experiments on two unseen domains that are not included in MMLU, namely Art [58] and Military [59], using Llama3-8B-Instruct. For a new query q_{new} , we obtain a latent representation $z_{\text{new}} \sim p'_{\theta}(z | q_{\text{new}})$ from the learned prior network and compute its cosine similarity to the centroids of the seven in-domain latent clusters identified in Fig. 4. Figure 5 shows that the learned prior maps unseen queries to semantically plausible regions of the latent space by associating them with related in-domain clusters, such as placing Art closer to *Humanities & History*. This behavior suggests that the topology initializer can produce plausible collaboration graphs even for unseen domains, which is consistent with the observed gains in out-of-domain performance.

H. Low-Resource Learning Analysis

To evaluate the robustness of TopoPrior in data-scarce domains, we conduct a low-resource analysis to assess whether TopoPrior can effectively leverage limited domain data. We compare the results of ARG-Designer with and without TopoPrior under several low-resource regimes on MMLU. Specifically, we subsample the original training data to 5%, 10%, and 20% of the full size in each domain. The TopoPrior generator is trained using the same hyperparameters as in the full-data setting and subsequently evaluated on the corresponding test splits.

Table IX shows that TopoPrior improves the results of ARG-Designer across all data fractions, suggesting that it can transfer useful topological priors even under limited supervision. At the lowest data setting (5%), incorporating TopoPrior yields particularly clear gains (e.g., +1.04 points in “Humanities & History” and +2.58 points in “Ethics & Morality”), indicating that the learned prior can provide structural guidance when labeled examples are scarce. Despite fluctuations in absolute performance, TopoPrior maintains a positive improvement margin across all training-data sizes.

TABLE IX
Low-resource training results over MMLU of TopoPrior + ARG-Designer in terms of accuracy (%).

Domain	With TopoPrior?		Δ	With TopoPrior?		Δ	With TopoPrior?		Δ
	No	Yes		No	Yes		No	Yes	
Training data	5% of the original			10% of the original			20% of the original		
Natural Science	60.34	61.95	1.61↑	61.89	62.75	0.86↑	62.64	64.96	2.32↑
Engineering & Technology	63.46	64.80	1.34↑	64.72	66.05	1.33↑	65.94	67.58	1.64↑
Social Sciences	58.25	60.86	2.61↑	60.13	63.31	3.18↑	61.08	65.42	4.34↑
Humanities & History	62.88	63.92	1.04↑	64.71	65.83	1.12↑	66.15	67.89	1.74↑
Law, Government, Public Affairs	55.13	57.64	2.51↑	58.55	60.07	1.52↑	60.41	63.32	2.91↑
Ethics & Morality	53.66	56.24	2.58↑	56.85	58.70	1.85↑	59.19	60.61	1.42↑
Business & Management	59.40	61.88	2.48↑	61.23	63.65	2.42↑	62.90	65.77	2.87↑
Average	59.02	61.04	2.02↑	61.15	62.91	1.76↑	62.62	65.08	2.46↑

TABLE X
Performance on niche domains.

Model	Astro-QA		TREC Genomics	
	Acc. (%)	Oracle Gap	Acc. (%)	Oracle Gap
ARG-Designer	71.4	-3.7	61.8	-3.3
AutoGen	68.2	-5.9	60.6	-4.5
Oracle	74.1	-	65.1	-
TopoPrior+ARG	73.5	-0.6	64.3	-0.8

These results suggest that the learned latent space remains useful in low-resource settings.

I. Fixed Agent Pool Domain Generalization

We next discuss why a fixed role pool may still generalize to niche and out-of-distribution (OOD) domains. The key intuition is that semantic similarity between queries and roles, together with the compositional flexibility of collaboration graphs, can enable role reuse beyond the domains explicitly represented in the training partition. We support this perspective with both conceptual discussion and empirical evidence.

As shown in Table II, the TopoPrior role pool spans broad domain categories (e.g., Natural Sciences, Engineering, and Social Sciences), each comprising multiple subdomains. Niche domains can often be associated with these broader categories. For example, a genetics query falls under Natural Sciences/Biology and can be addressed by both the “Natural Science Expert” and “Medical Life Scientist” roles. Crucially, TopoPrior does not assign roles statically; rather, it composes multiple roles and their interactions according to the content and context of each query. For a genetics question, the generated graph may include (1) a Natural Science Expert for core biological knowledge, (2) a Medical Life Scientist when the question involves genetic disorders, (3) a Chemistry Specialist for molecular genetics topics such as DNA structure, and (4) a General Coordinator to orchestrate collaboration. Hence, even without an explicit “Genetics Expert” role, a combination of existing roles may still address the query effectively. The learned initializer adapts role composition to the query context, enabling a degree of generalization through semantic coverage.

1) *Domain Validation of Generalization:* To further examine whether a fixed role pool can remain useful in niche domains, we construct two niche-domain test sets: Astro-QA [60]

TABLE XI
Token cost analysis for training and inference.

Metric	Value
Avg. tokens per training graph (ARG)	~ 1,200
Total training tokens (~ 100k samples)	~ 120M
Avg. inference tokens per test query (ARG alone)	~ 800
Avg. inference tokens per test query (TopoPrior+ARG)	~ 478
Token savings per test query	322
Total inference savings (MMLU test set)	4.83M
Break-even test queries needed	120M / 322 ≈ 373,670

for astrophysics and TREC Genomics [61] for genetics. These domains are not explicitly represented in the original role pool. Specifically, astrophysics can be viewed as a subdomain of Natural Sciences/Physics, while genetics falls under Natural Sciences/Biology/Medicine. We compare three configurations: (1) ARG-Designer and AutoGen [31], which dynamically generate roles from scratch; (2) TopoPrior+ARG, which uses the fixed role pool; and (3) Oracle, in which a dedicated domain-specific role is added to the pool as a reference upper bound. Table X shows that TopoPrior+ARG performs within 1% of the Oracle, suggesting that semantic role composition can provide useful coverage for niche domains. By contrast, the dynamic role-generation baselines (ARG-Designer and AutoGen) perform worse in this setting, suggesting that reusing semantically related roles from a fixed pool may be more effective than generating roles from scratch.

J. Analysis of End-to-End Token Cost

The one-time cost of generating reference collaboration graphs for training TopoPrior may be amortized by token savings during inference, which can make the overall pipeline cost-effective at sufficiently large deployment scales. Reference graphs are generated only once during offline training. After training, the TopoPrior generator can produce initial collaboration topologies for any number of test queries without re-running the expensive topology-evolution algorithm. For any domain, the supervision cost scales with the training set size, whereas inference benefits from a **40.2%** token reduction (see Section IV-F) per test query. Token savings therefore accumulate as more test queries are processed and may eventually offset the initial offline cost.

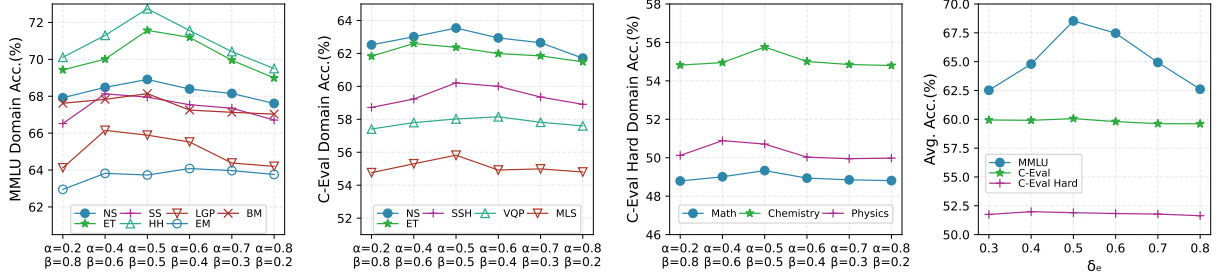


Fig. 6: Hyperparameter analysis of the loss coefficients α , β , and edge generation threshold δ_e . “NS”, “ET”, “SS”, “HH”, “LGP”, “EM” and “BM” are abbreviations for different domains in MMLU and C-Eval, respectively.

We measure the total training token cost for graph generation on the MMLU training set (seven domains, $\sim 100k$ samples) using ARG-Designer, and compute total inference savings using TopoPrior+ARG on the MMLU test set (15k samples), extrapolating to larger test sets. As shown in Table XI, the MMLU test set alone does not recoup the training cost (4.83M saved tokens versus 120M tokens invested). However, at larger deployment scales, the break-even point is reached after approximately **373,670** queries. Once TopoPrior is trained, it can be reused across domains and datasets without additional graph generation, which further improves amortization in repeated-use scenarios.

K. Hyperparameter Analysis

We analyze key hyperparameters, including the loss coefficients α and β , as well as the edge-generation threshold δ_e , across multiple dataset subdomains using Llama3-8B-Instruct. Figure 6 reports the average performance over these settings. The best hyperparameter combination is used for the main experimental results.

V. CONCLUSION

In this paper, we presented TopoPrior, a framework for transferable topology prior learning for multi-agent LLM collaboration in multi-domain settings. Rather than constructing collaboration graphs from scratch for each task, TopoPrior learns reusable, query-conditioned topology priors from reference collaboration graphs and uses them to initialize downstream topology-evolution backbones. Experiments on MMLU, C-Eval, and additional out-of-domain settings show that TopoPrior improves downstream reasoning performance in the evaluated settings while reducing online communication cost, token usage, and refinement rounds. These results suggest that transferable topology-prior learning is a practical direction for improving the efficiency and adaptability of multi-agent LLM systems.

Several directions remain for future work. One is to reduce reliance on expensive offline supervision by using cheaper or more weakly supervised reference graphs. Another is to extend the current predefined role pool toward more open-ended role discovery and role composition. It is also promising to investigate tighter integration between topology-prior learning and downstream topology refinement in interactive reasoning settings. From a broader-impact perspective, TopoPrior may improve the efficiency and accessibility of multi-agent

LLM systems by reducing communication overhead and improving collaboration quality. At the same time, such systems should be deployed with appropriate evaluation, transparency, and human oversight, especially in high-stakes applications.

REFERENCES

- [1] N. Nazaryova, S. Chahed, T. Chausalet, and M. Dwek, “Leveraging large language models for medical text classification: a hospital readmission prediction case,” in *ICPRS*, 2024, pp. 1–7.
- [2] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, “Measuring massive multitask language understanding,” in *Proc. Int. Conf. Learn. Represent.*, 2021.
- [3] Y. Huang, Y. Bai, Z. Zhu, J. Zhang, J. Zhang, T. Su, J. Liu, C. Lv, Y. Zhang, J. Lei, Y. Fu, M. Sun, and J. He, “C-eval: A multi-level multi-discipline chinese evaluation suite for foundation models,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2023.
- [4] M. Nghiem, P. Baylis, A. Freitas, and S. Ananiadou, “Text classification and prediction in the legal domain,” in *LREC*, 2022, pp. 4717–4722.
- [5] H. Lee, K. C. Li, M. Grabmair, and S. Xu, “Efficient prompt optimisation for legal text classification with proxy prompt evaluator,” *CoRR*, vol. abs/2510.08524, 2025.
- [6] H. Chen, X. Han, Z. Wu, and Y. Jiang, “Multi-prompt alignment for multi-source unsupervised domain adaptation,” in *Proc. Adv. Neural Inf. Process. Syst.*, 2023.
- [7] H. Nguyen, Y. Liu, C. Zhang, T. Zhang, and P. S. Yu, “Cof-cot: Enhancing large language models with coarse-to-fine chain-of-thought prompting for multi-domain NLU tasks,” in *Proc. Conf. Empir. Methods Nat. Lang. Process.*, 2023, pp. 12 109–12 119.
- [8] T. Hu, P. Zhang, B. Yang, J. Xie, D. F. Wong, and R. Wang, “Large language model for multi-domain translation: Benchmarking and domain cot fine-tuning,” in *Proc. Conf. Empir. Methods Nat. Lang. Process.*, 2024, pp. 5726–5746.
- [9] Q. Zhang, D. Wang, H. Qian, Y. Li, T. Zhang, M. Huang, K. Xu, H. Li, L. Yan, and H. Qiu, “Understanding the dark side of llms’ intrinsic self-correction,” in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2025, pp. 27 066–27 101.
- [10] T. Scialom, T. Chakrabarty, and S. Muresan, “Fine-tuned language models are continual learners,” in *Proc. Conf. Empir. Methods Nat. Lang. Process.*, 2022, pp. 6107–6122.
- [11] Z. Ling, D. Chen, L. Yao, Y. Li, and Y. Shen, “Diversity as a reward: Fine-tuning llms on a mixture of domain-undetermined data,” *CoRR*, vol. abs/2502.04380, 2025.
- [12] J. Li, Z. Yu, Z. Du, L. Zhu, and H. T. Shen, “A comprehensive survey on source-free domain adaptation,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 46, no. 8, pp. 5743–5762, 2024.
- [13] X. Liang, L. Yang, J. Wang, Y. Lu, R. Wu, H. Chen, and J. Hao, “Boosting multi-domain fine-tuning of large language models through evolving interactions between samples,” in *Proc. Int. Conf. Mach. Learn.*, 2025.
- [14] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, “Metagpt: Meta programming for A multi-agent collaborative framework,” in *Proc. Int. Conf. Learn. Representation*, 2024.
- [15] Z. Liu, Y. Zhang, P. Li, Y. Liu, and D. Yang, “A dynamic llm-powered agent network for task-oriented agent collaboration,” *CoRR*, vol. abs/2310.02170, 2024.
- [16] G. Zhang, Y. Yue, Z. Li, S. Yun, G. Wan, K. Wang, D. Cheng, J. X. Yu, and T. Chen, “Cut the crap: An economical communication pipeline for llm-based multi-agent systems,” in *Proc. Int. Conf. Learn. Repr.*, 2025.

- [17] G. Zhang, Y. Yue, X. Sun, M. Yu, K. Wang, T. Chen, and D. Cheng, "G-designer: Architecting multi-agent communication topologies via graph neural networks," in *Proc. Int. Conf. Mach. Learn.*, 2025.
- [18] H. Li, A. Wang, K. Li, Z. Wang, L. Zhang, D. Qiu, Q. Liu, and J. Su, "A multi-agent framework with automated decision rule optimization for cross-domain misinformation detection," in *Proc. Conf. Empir. Methods Nat. Lang. Process.*, Nov. 2025.
- [19] W. Kwan, X. Zeng, Y. Wang, Y. Sun, L. Li, Y. Jiang, L. Shang, Q. Liu, and K. Wong, "M4LE: A multi-ability multi-range multi-task multi-domain long-context evaluation benchmark for large language models," in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2024, pp. 15 568–15 592.
- [20] Q. Chen, L. Qin, J. Zhang, Z. Chen, X. Xu, and W. Che, "M³cot: A novel benchmark for multi-domain multi-step multi-modal chain-of-thought," in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2024, pp. 8199–8221.
- [21] N. Nikolaidis, N. Stefanovitch, P. Silvano, D. I. Dimitrov, R. Yangarber, N. Guimarães, E. Sartori, I. Androutsopoulos, P. Nakov, G. D. S. Martino, and J. Piskorski, "Polynarrative: A multilingual, multilabel, multi-domain dataset for narrative extraction from news articles," in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2025, pp. 31 323–31 345.
- [22] Z. Liu, S. Huang, T. Guo, M. Hou, and Q. Liang, "A prompt-driven framework for multi-domain knowledge tracing," *Mach. Learn.*, vol. 114, no. 4, Feb. 2025.
- [23] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "Lora: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learn. Representation*, 2022.
- [24] P. Schaffhalter, S. Liao, Y. Zhou, C. Yeh, A. Kandoor, and J. Laudon, "Scalable multi-domain adaptation of language models using modular experts," *CoRR*, vol. abs/2410.10181, 2024.
- [25] J. Li, B. Wang, X. Zhou, and X. Hu, "Dynamic expert specialization: Towards catastrophic forgetting-free multi-domain moe adaptation," *CoRR*, vol. abs/2509.16882, 2025.
- [26] D. Jiang, X. Ren, and B. Y. Lin, "Llm-blender: Ensembling large language models with pairwise ranking and generative fusion," in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2023, pp. 14 165–14 178.
- [27] J. Zhang, X. Xu, N. Zhang, R. Liu, B. Hooi, and S. Deng, "Exploring collaboration mechanisms for LLM agents: A social psychology view," in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2024, pp. 14 544–14 607.
- [28] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, "Improving factuality and reasoning in language models through multiagent debate," in *Proc. Int. Conf. Mach. Learn.*, 2024.
- [29] S. Holt, M. R. Luyten, and M. van der Schaar, "L2MAC: large language model automatic computer for unbounded code generation," *CoRR*, vol. abs/2310.02003, 2023.
- [30] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "Chatdev: Communicative agents for software development," in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2024, pp. 15 174–15 186.
- [31] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, "Autogen: Enabling next-gen LLM applications via multi-agent conversations," in *Proc. Conf. Lang. Model.*, 2024.
- [32] Z. Zhou, B. Hu, P. Zhang, C. Zhao, and B. Liu, "Large language model is a good policy teacher for training reinforcement learning agents," *CoRR*, vol. abs/2311.13373, 2023.
- [33] Y. Ishibashi and Y. Nishimura, "Self-organized agents: A LLM multi-agent framework toward ultra large-scale code generation and optimization," *CoRR*, vol. abs/2404.02183, 2024.
- [34] M. Zhuge, W. Wang, L. Kirsch, F. Faccio, D. Khizbullin, and J. Schmidhuber, "Gptswarm: Language agents as optimizable graphs," in *Proc. Int. Conf. Mach. Learn.*, 2024.
- [35] Z. Wang, Y. Wang, X. Liu, L. Ding, M. Zhang, J. Liu, and M. Zhang, "Agentdropout: Dynamic agent elimination for token-efficient and high-performance llm-based multi-agent collaboration," in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2025, pp. 24 013–24 035.
- [36] S. Li, Y. Liu, Q. Wen, C. Zhang, and S. Pan, "Assemble your crew: Automatic multi-agent communication topology design via autoregressive graph generation," in *Proc. Assoc. for the Advan. of Arti. Intell.*, 2026, pp. 23 142–23 150.
- [37] D. Dua, E. Strubell, S. Singh, and P. Verga, "To adapt or to annotate: Challenges and interventions for domain adaptation in open-domain question answering," in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2023, pp. 14 429–14 446.
- [38] G. Dey and Y. K. Lal, "On the transferability of causal knowledge for language models," in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2025, pp. 8–14.
- [39] T. N. Kipf and M. Welling, "Variational graph auto-encoders," *CoRR*, vol. abs/1611.07308, 2016.
- [40] Y. Cao, S. Han, Z. Gao, Z. Ding, X. Xie, and S. K. Zhou, "GraphInsight: Unlocking insights in large language models for graph structure understanding," in *Proc. Annu. Meeting Assoc. Comput. Linguistics*, 2025, pp. 12 096–12 134.
- [41] D. Goswami, R. Schuster, J. van de Weijer, and D. Stricker, "Attribution-aware weight transfer: A warm-start initialization for class-incremental semantic segmentation," in *Proc. Winter Conf. Appl. Comput. Vis.*, 2023, pp. 3194–3203.
- [42] R. Angell and A. McCallum, "Fast, scalable, warm-start semidefinite programming with spectral bundling and sketching," in *Proc. Int. Conf. Mach. Learn.*, 2024, pp. 1579–1615.
- [43] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Representation*, 2017.
- [44] J. Devlin, M. Chang, K. Lee, and K. Toutanova, "BERT: pre-training of deep bidirectional transformers for language understanding," in *Proc. Annu. Conf. North Am. Chapter Assoc. Comput. Linguist.*, 2019, pp. 4171–4186.
- [45] Y. Ganin and V. S. Lempitsky, "Unsupervised domain adaptation by backpropagation," in *Proc. Int. Conf. Mach. Learn.*, 2015, pp. 1180–1189.
- [46] H. Zhao, R. T. des Combes, K. Zhang, and G. J. Gordon, "On learning invariant representations for domain adaptation," in *Proc. Int. Conf. Mach. Learn.*, 2019, pp. 7523–7532.
- [47] A. Sicilia, K. Atwell, M. Alikhani, and S. J. Hwang, "Pac-bayesian domain adaptation bounds for multiclass learners," in *Proc. Conf. Uncertainty Artif. Intell.*, 2022, pp. 1824–1834.
- [48] Y. E. Nesterov, *Introductory Lectures on Convex Optimization - A Basic Course*, ser. Applied Optimization. Springer, 2004, vol. 87.
- [49] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "Llama: Open and efficient foundation language models," 2023.
- [50] A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, H. Lin, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Lin, K. Dang, K. Lu, K. Bao, K. Yang, L. Yu, M. Li, M. Xue, P. Zhang, Q. Zhu, R. Men, R. Lin, T. Li, T. Xia, X. Ren, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Wan, Y. Liu, Z. Cui, Z. Zhang, and Z. Qiu, "Qwen2.5 technical report," *CoRR*, vol. abs/2412.15115, 2024.
- [51] DeepSeek-AI, "Deepseek-v3 technical report," 2025.
- [52] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022.
- [53] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proc. Adv. Neural Inf. Process. Syst.*, 2020.
- [54] Y. Ma, Z. Liang, H. Dai, B. Chen, D. Gao, Z. Ran, Z. Wang, L. Jin, W. Jiang, G. Zhang, X. Cai, and L. Yang, "Modula: Mixture of domain-specific and universal lora for multi-task learning," in *Proc. Conf. Empir. Methods Nat. Lang. Process.*, 2024, pp. 2758–2770.
- [55] A. F. Agarap, "Deep learning using rectified linear units (relu)," *CoRR*, vol. abs/1803.08375, 2018.
- [56] J. Chung, Ç. Gülçehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," *CoRR*, vol. abs/1412.3555, 2014.
- [57] L. van der Maaten and G. Hinton, "Visualizing data using t-sne," *Journal of Machine Learning Research*, vol. 9, no. 86, pp. 2579–2605, 2008.
- [58] N. Garcia and G. Vogiatzis, "How to read paintings: Semantic art understanding with multi-modal retrieval," in *Proc. Eur. Conf. Comput. Vis.*, 2018, pp. 676–691.
- [59] M. Zhu, Z. Xu, K. Zeng, K. Xiao, M. Wang, W. Ke, and H. Huang, "CMNEE: A large-scale document-level event extraction dataset based on open-source chinese military news," in *Proc. Int. Conf. Comput. Linguist.*, 2024, pp. 3367–3379.
- [60] J. Li, F. Zhao, P. Chen, J. Xie, X. Zhang, H. Li, M. Chen, Y. Wang, and M. Zhu, "An astronomical question answering dataset for evaluating large language models," *Nature Scientific Data*, 2025.
- [61] W. R. Hersh, A. M. Cohen, J. Yang, R. T. Bhupatiraju, P. M. Roberts, and M. A. Hearst, "Trec 2007 genomics track overview," in *Text Retrieval Conference*, 2007.

APPENDIX A PROOFS OF THEORETICAL RESULTS

In this appendix, we provide proofs and supporting statements for the analytical results in Section III-E. The analysis formalizes two core intuitions underlying TopoPrior: (i) latent-space alignment can support cross-domain generalization, and (ii) improved topology initialization can reduce the number of refinement rounds and the associated token cost.

Let $\mathcal{G}$ denote the space of directed labeled graphs corresponding to agent collaboration topologies. For any query q , domain label d , and ground-truth answer y , let $p^*(\cdot | q, d)$ denote an unknown high-utility distribution over collaboration graphs for that query and domain. Our learned generator is denoted by $p_\theta(\cdot | q)$. When executed with a graph G , the downstream multi-agent system produces a random output $\hat{y}$ with distribution $\pi_{\text{MAS}}(\cdot | q, G)$. The task loss $\ell(\hat{y}, y) \in [0, 1]$ measures the discrepancy between the system output and the ground truth.

We define the *accuracy–token utility* as

$$J_\lambda(G; q, y) = 1 - \mathbb{E}_{\hat{y} \sim \pi_{\text{MAS}}(\cdot | q, G)}[\ell(\hat{y}, y)] - \lambda \tilde{C}(G; q),$$

where $\tilde{C}(G; q) = C(G; q)/C_{\max}$ is the normalized token cost and $\lambda \geq 0$ is a trade-off coefficient between correctness and communication cost. Since $\ell(\hat{y}, y) \in [0, 1]$ and $\tilde{C}(G; q) \geq 0$, the utility J_λ takes values in $[-\lambda, 1]$.

A. Proofs for Section III-E1: Cross-Domain Transfer via Latent Alignment

Let Z be the latent space induced by the variational encoder. For each source domain $k \in \{1, \dots, K\}$, define the marginal latent distribution

$$P_k(z) = \mathbb{E}_{(q, y) \sim \mathcal{D}_k} \mathbb{E}_{G^* \sim p^*(\cdot | q, k)}[q_\phi(z | q, G^*)],$$

where $q_\phi(z | q, G)$ denotes the encoder distribution. For a target domain t , we analogously define $P_t(z)$.

We consider a hypothesis class $\mathcal{H}$ of functions $h : Z \rightarrow \mathcal{Y}$. For a hypothesis h , the expected error on domain d is

$$\epsilon_d(h) = \mathbb{E}_{z \sim P_d}[\ell_h(z)],$$

where ℓ_h is a bounded loss.

To quantify discrepancy between latent distributions, we use the $\mathcal{H}\Delta\mathcal{H}$ -divergence.

Definition 3 ($\mathcal{H}\Delta\mathcal{H}$ -divergence). *For two distributions P and Q on Z ,*

$$d_{\mathcal{H}\Delta\mathcal{H}}(P, Q) = 2 \sup_{h, h' \in \mathcal{H}} \left| \Pr_{z \sim P}[h(z) \neq h'(z)] - \Pr_{z \sim Q}[h(z) \neq h'(z)] \right|.$$

This divergence measures the largest change in pairwise hypothesis disagreement between the two distributions.

Proof of Theorem 1. Let $\alpha = (\alpha_1, \dots, \alpha_K)$ be convex weights such that $\alpha_k \geq 0$ and $\sum_{k=1}^K \alpha_k = 1$. Define the source-mixture distribution

$$P_\alpha = \sum_{k=1}^K \alpha_k P_k$$

and the corresponding weighted source error

$$\epsilon_\alpha(h) = \sum_{k=1}^K \alpha_k \epsilon_k(h).$$

Let

$$h^* \in \arg \min_{h \in \mathcal{H}} (\epsilon_\alpha(h) + \epsilon_t(h)),$$

and define

$$\lambda_{\alpha, t}^* = \epsilon_\alpha(h^*) + \epsilon_t(h^*).$$

Starting from

$$\epsilon_t(h) = \epsilon_\alpha(h) + (\epsilon_t(h) - \epsilon_t(h^*)) + (\epsilon_t(h^*) + \epsilon_\alpha(h^*)) + (\epsilon_\alpha(h^*) - \epsilon_\alpha(h)),$$

we obtain

$$\epsilon_t(h) \leq \epsilon_\alpha(h) + |\epsilon_t(h) - \epsilon_t(h^*) - (\epsilon_\alpha(h) - \epsilon_\alpha(h^*))| + \lambda_{\alpha, t}^*.$$

Using the standard multi-source domain adaptation bound based on $\mathcal{H}\Delta\mathcal{H}$, the absolute term is bounded by

$$\frac{1}{2} d_{\mathcal{H}\Delta\mathcal{H}}(P_\alpha, P_t).$$

Therefore,

$$\epsilon_t(h) \leq \epsilon_\alpha(h) + \frac{1}{2} d_{\mathcal{H}\Delta\mathcal{H}}(P_\alpha, P_t) + \lambda_{\alpha, t}^*.$$

Substituting $\epsilon_\alpha(h) = \sum_{k=1}^K \alpha_k \epsilon_k(h)$ concludes the proof. $\square$

a) Interpretation.: Theorem 1 bounds the target-domain error by three terms: the weighted source-domain error, the latent-space divergence between the source mixture and the target domain, and the best joint error achievable by the hypothesis class. In TopoPrior, the adversarial regularizer is intended to reduce latent-space domain discrimination, thereby lowering divergence and supporting transfer to unseen domains.

B. Proofs for Section III-E2: Topology Initialization as Search Acceleration

We now analyze how improved initialization can reduce refinement rounds and token cost. Let

$$U_t = \mathbb{E}[J_\lambda(G_t; q, y)]$$

be the expected utility at evolution step t , starting from G_0 , and let U^* denote the optimal utility achievable by the evolution procedure.

Proof of Theorem 2. Under Assumption 1, for all t ,

$$U^* - U_{t+1} \leq (1 - \eta)(U^* - U_t).$$

Recursively applying this inequality for T steps gives

$$U^* - U_T \leq (1 - \eta)^T (U^* - U_0).$$

To ensure $U^* - U_T \leq \epsilon$, it suffices to require

$$(1 - \eta)^T (U^* - U_0) \leq \epsilon,$$

which is equivalent to

$$T \geq \frac{\log((U^* - U_0)/\epsilon)}{\log(1/(1 - \eta))},$$

since $0 < 1 - \eta < 1$. This completes the proof. $\square$

Proof of Corollary 1. Applying Theorem 2 to prior-based and scratch initialization gives

$$T_{\text{prior}}(\epsilon) = \frac{\log((U^* - U_0^{\text{prior}})/\epsilon)}{\log(1/(1 - \eta))},$$

$$T_{\text{scratch}}(\epsilon) = \frac{\log((U^* - U_0^{\text{scratch}})/\epsilon)}{\log(1/(1 - \eta))}.$$

Subtracting yields

$$T_{\text{prior}}(\epsilon) - T_{\text{scratch}}(\epsilon) = \frac{\log\left(\frac{U^* - U_0^{\text{prior}}}{U^* - U_0^{\text{scratch}}}\right)}{\log(1/(1 - \eta))}.$$

Since $U_0^{\text{prior}} > U_0^{\text{scratch}}$, the numerator is negative and the denominator is positive, so $T_{\text{prior}}(\epsilon) < T_{\text{scratch}}(\epsilon)$. $\square$

Next, we formalize token-cost savings from fewer refinement rounds and sparser initial graphs.

Assumption 2 (Token cost bounded by graph size). *There exist constants $a_V, a_E, b \geq 0$ such that for any graph G and query q ,*

$$C(G; q) \leq a_V|V(G)| + a_E|E(G)| + b.$$

Assumption 3 (Bounded graph size during evolution). *For all steps t before termination,*

$$\mathbb{E}[|V(G_t)| + |E(G_t)|] \leq M$$

for some constant $M \geq 1$.

Proposition 1 (Total expected token cost over T rounds). *Under Assumptions 2 and 3,*

$$\mathbb{E}\left[\sum_{t=0}^{T-1} C(G_t; q)\right] \leq T((a_V + a_E)M + b).$$

Proof. From Assumption 2, we have

$$C(G_t; q) \leq a_V|V(G_t)| + a_E|E(G_t)| + b.$$

Taking expectations on both sides and using Assumption 3, we obtain

$$\mathbb{E}[C(G_t; q)] \leq a_V\mathbb{E}[|V(G_t)|] + a_E\mathbb{E}[|E(G_t)|] + b \leq (a_V + a_E)M + b.$$

Summing over $t = 0, \dots, T - 1$ establishes the result. $\square$

Corollary 2 (Token savings from fewer rounds). *If TopoPrior reduces the number of rounds from T_{scratch} to T_{prior} , then the reduction in the bound of Proposition 1 is*

$$(T_{\text{scratch}} - T_{\text{prior}})((a_V + a_E)M + b).$$

Assumption 4 (Token cost monotonicity in graph size). *For fixed q , $C(G; q)$ is non-decreasing in both $|V(G)|$ and $|E(G)|$.*

Proposition 2 (Sparser initialization reduces first-round cost). *Under Assumption 4, if the prior-initialized graph G_0^{prior} satisfies*

$$\mathbb{E}[|V(G_0^{\text{prior}})|] \leq \mathbb{E}[|V(G_0^{\text{scratch}})|],$$

$$\mathbb{E}[|E(G_0^{\text{prior}})|] \leq \mathbb{E}[|E(G_0^{\text{scratch}})|],$$

then

$$\mathbb{E}[C(G_0^{\text{prior}}; q)] \leq \mathbb{E}[C(G_0^{\text{scratch}}; q)].$$

Proof. By Assumption 4, smaller or equal node and edge counts imply no greater token cost. Taking expectations preserves the inequality. $\square$

a) Interpretation.: These results highlight two complementary efficiency effects. First, under Assumption 1, better initialization reduces the number of refinement rounds needed to achieve a target utility. Second, if the initialized graph is also sparser, the per-round token cost decreases. Together, these results provide analytical support for the efficiency trends observed in our experiments.